Современные сервисные платформенные решения.

Лекция 7. Интернет-технологии

Старший преподаватель кафедры
«Информационные системы»

Шеркунов Вячеслав Владимирович



Ульяновский государственный
технический университет

2025

Содержание лекции

- JSON
- HTTP-сервисы (REST)
- HTTP-запросы
- Web-сервисы

JSON

JSON (JavaScript Object Notation) – это текстовый формат обмена данными, с которым могут работать все браузеры. Этот формат похож на XML, но по сравнению с XML он является более лаконичным и требует меньше места. В настоящее время, данный формат обмена данными широко используется в веб-приложениях.

Есть несколько причин для широкого использования этого формата на уровне платформы:

- Во-первых, JSON – это современный формат, с помощью которого прикладные решения «1С:Предприятия» могут осуществлять интеграцию со сторонними приложениями.
- Во-вторых, JSON активно используется в HTTP-интерфейсах, а платформа «1С:Предприятие 8» как раз предоставляет два способа реализации таких интерфейсов – это REST-интерфейс, который автоматически формируется для всего прикладного решения, и HTTP-сервисы, которые можно создавать самостоятельно.

В платформе реализовано несколько слоев работы с JSON. Самые универсальные и гибкие – это низкоуровневые средства потоковой записи и чтения. Более высокоуровневые и не такие универсальные – средства сериализации в JSON примитивных типов и коллекций «1С:Предприятия». И, наконец, третий слой – это средства, позволяющие сериализовать/десериализовать прикладные типы «1С:Предприятия»: ссылки, объекты, наборы записей и вообще любые типы, для которых поддерживается XDTO-сериализация.

JSON

JSON – это текстовый формат, поэтому данные в формате JSON могут содержать:

- *Объект* – неупорядоченное множество пар <имя свойства>:<значение>, заключенный в фигурные скобки ({}). Пары <имя свойства>:<значение> разделяются запятыми (,).
- *Массив* – множество значений. Массив заключается в квадратные скобки ([]). Значения разделяются запятыми (,).
- *Значение* – может быть строкой, числом, объектом, массивом или литералом true, false, null.
- *Строка* – набор символов, заключенный в двойные кавычки ("").
- *Число* – сериализуется с разделителем точка (.). Точность числа не ограничена.

Таким образом, с помощью вышеперечисленных элементов допускается описание объектов любой сложности для представления в формате JSON.

JSON

Существует несколько основных сценариев использования JSON:

- Интеграция с внешними системами через их HTTP-интерфейсы: Google Calendar, Salesforce.com, REST-интерфейс «1С:Предприятия», SharePoint и т. д.
- Организация собственного HTTP-интерфейса прикладного решения.
- Обмен файлами JSON с внешними системами. Формирование конфигурационных, настроечных файлов. Использование их в процедурах обмена данными, например с интернет-магазинами.
- Использование файлов JSON для обмена данными между разными приложениями «1С:Предприятия».

Потоковая работа с JSON

Для потоковой работы предназначены объекты ЧтениеJSON и ЗаписьJSON. Они последовательно, от значения к значению, читают JSON из файла или строки или последовательно записывают JSON в файл или строку. Таким образом, чтение и запись JSON происходят без формирования всего документа в памяти. В результате потоковая техника может дать существенный выигрыш, особенно если объем документа потенциально большой или же если этот объем заранее неизвестен.

Для того чтобы выполнить потоковую запись JSON-документа, необходимы записываемые данные и объект *ЗаписьJSON*.

Допустим, нам нужно записать в JSON-документ некоторый объект, обладающий свойствами *Код*, *Наименование*, *Телефоны* (массив строк), *ОбъемПродаж*, *Поставщик*.

Один JSON-файл содержит одно значение. Это то самое значение, о котором говорилось выше: объект, массив, строка, число или один из литералов.

Потоковая работа с JSON

```
&НаСервереБезКонтекста
Процедура ПотоковаяЗаписьНаСервере()

    // Создать объект записи и открыть файл, в который
    будет выполняться запись.
    Запись = Новый ЗаписьJSON;
    Запись.ОткрытьФайл("c:\temp\streamWrite_2.json", , ,
    Новый ПараметрыЗаписиJSON(, Символы.Таб));

    // Выполнить запись значений с помощью объекта записи
    (Запись).
    // Записать начало корневого объекта.
    Запись.ЗаписатьНачалоОбъекта();

    // Записать имя свойства корневого объекта.
    Запись.ЗаписатьИмяСвойства("Контрагент1");

    // Записать начало нашего объекта.
    Запись.ЗаписатьНачалоОбъекта();

    // Заполнить свойство Код типа Строка.
    Запись.ЗаписатьИмяСвойства("Код");
    Запись.ЗаписатьЗначение("000000017");

    // Заполнить свойство Наименование типа Строка.
    Запись.ЗаписатьИмяСвойства("Наименование");
    Запись.ЗаписатьЗначение("ОАО Топаз");

    // Заполнить свойство Телефоны типа Массив.
    // Поэтому после имени свойтва записываем массив,
    состоящий из значений - строк.
    Запись.ЗаписатьИмяСвойства("Телефоны");
    Запись.ЗаписатьНачалоМассива();
    Запись.ЗаписатьЗначение("8-999-777-55-33");
    Запись.ЗаписатьЗначение("+71112223344");
    Запись.ЗаписатьКонецМассива();

    // Заполнить свойство ОбъемПродаж типа Число.
    Запись.ЗаписатьИмяСвойства("ОбъемПродаж");
    Запись.ЗаписатьЗначение(5000000);

    // Заполнить свойство Поставщик типа Булево.
    Запись.ЗаписатьИмяСвойства("Поставщик");
    Запись.ЗаписатьЗначение(Ложь);

    // Записать конец нашего объекта.
    Запись.ЗаписатьКонецОбъекта();

    // Записать конец корневого объекта.
    Запись.ЗаписатьКонецОбъекта();

    // Завершить работу с файлом.
    Запись.Закрыть();

КонецПроцедуры
```

Потоковая работа с JSON

В результате сформированный JSON-документ будет иметь следующий вид:

```
{
  "Контрагент1": {
    "Код": "000000017",
    "Наименование": "ОАО Топаз",
    "Телефоны": [
      "8-999-777-55-33",
      "+71112223344"
    ],
    "ОбъемПродаж": 5000000,
    "Поставщик": false
  }
}
```

Потоковая работа с JSON

Потоковое чтение JSON-документа выполняется с помощью объекта ЧтениеJSON. Документ открывается для чтения и поэлементно считывается в цикле. При этом разработчик определяет, что считано, и соответствующим образом обрабатывает считываемые данные.

```
&НаСервереБезКонтекста
Процедура ПотоковоеЧтениеНаСервере()

	Сообщение = Новый СообщениеПользователю;

	// Создать объект чтения и открыть файл, из
	которого будет выполняться чтение.
	Чтение = Новый ЧтениеJSON;
	Чтение.ОткрытьФайл("c:\temp\streamWrite_2.jso
	n");

	// Выполнить чтение поэлементно в цикле.
	Пока Чтение.Прочитать() Цикл

	Если Чтение.ТипТекущегоЗначения = ТипЗначения
	JSON.ИмяСвойства Тогда
	Сообщение.Текст = "Имя =
	" + Чтение.ТекущееЗначение;
	Сообщение.Сообщить();
	КонецЕсли;

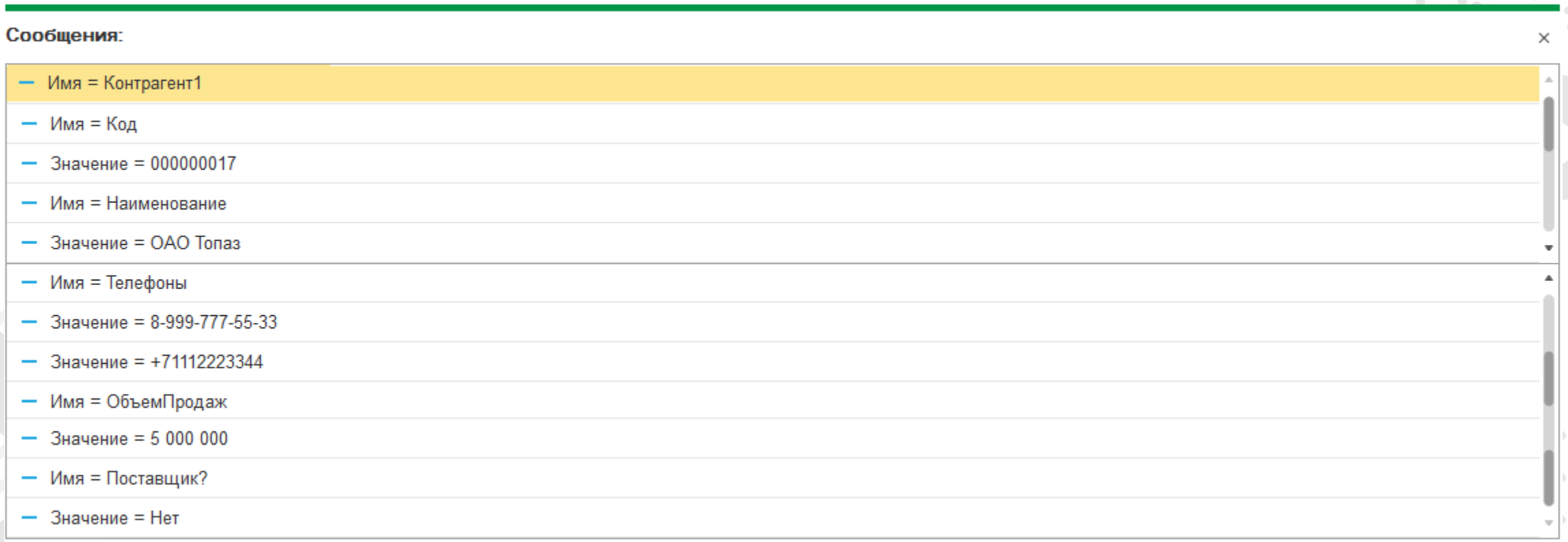
	Если Чтение.ТипТекущегоЗначения = ТипЗначения
	JSON.Булево Или
	Чтение.ТипТекущегоЗначения = ТипЗначенияJSON.
	Строка Или
	Чтение.ТипТекущегоЗначения = ТипЗначенияJSON.
	Число Или
	Чтение.ТипТекущегоЗначения = ТипЗначенияJSON.
	Комментарий Тогда
	Сообщение.Текст = "Значение =
	" + Чтение.ТекущееЗначение;
	Сообщение.Сообщить();
	КонецЕсли;
	КонецЦикла;

	// Завершить работу с файлом.
	Чтение.Закрыть();

КонецПроцедуры
```

Потоковая работа с JSON

Результат чтения JSON-документа будет выглядеть следующим образом:



Проверка структуры записываемого JSON-документа

Следует иметь в виду, что при потоковой записи разработчик самостоятельно формирует структуру JSON-документа. При записи данных объект ЗаписьJSON автоматически проверяет правильность записываемой структуры. За это отвечает свойство ПроверятьСтруктуру. По умолчанию оно истинно, поэтому если при записи нарушена целостность структуры JSON, то будет сгенерировано исключение и запись выполнена не будет. Но для увеличения скорости работы эту проверку можно отключить.

```
ЗаписьJSON.ПроверятьСтруктуру = Ложь;
```

HTTP-сервисы (REST)

Одним из распространенных способов взаимодействия приложений в Интернете является использование веб-сервисов. Одно приложение (поставщик веб-сервиса) публикует свой веб-сервис по уникальному веб-адресу в Интернете, а другие приложения (потребители веб-сервиса) могут обратиться к этому веб-сервису, для того чтобы получить какие-то данные от поставщика, передать ему какие-то данные или заставить поставщика выполнить какие-то действия.

Существуют разные протоколы и разные архитектуры, с помощью которых реализуются веб-сервисы. В платформе поддерживается работа с двумя видами веб-сервисов: веб-сервисы, реализованные в архитектуре REST (в «1С:Предприятии» они называются HTTP-сервисы), и веб-сервисы, работающие по протоколу SOAP (в «1С:Предприятии» они называются Web-сервисы).

HTTP-сервисы (REST)

HTTP-сервисы пользуются популярностью благодаря следующим качествам:

- простота программирования клиента таких сервисов;
- потенциально меньший объем передаваемых данных;
- потенциально меньшая вычислительная нагрузка;
- HTTP-сервисы ориентированы на «ресурсы», в то время как SOAP-сервисы ориентированы на «действия».

HTTP-сервисы (REST)

Обращение к HTTP-сервису выполняется по некоторому URL. В качестве примера можно привести HTTP-сервис, который при обращении по одному URL возвращает список каких-либо документов (например, расходных накладных), а при обращении по другому URL возвращает конкретную накладную. Действие, которое следует выполнить серверу, определяется тем, какой HTTP-метод адресован серверу. Например, если на сервер отправляется GET-запрос, то выполняется получение каких-либо данных, DELETE-запрос приводит к удалению данных и т. д.

При реализации HTTP-сервиса прикладной разработчик должен определить следующие элементы сервиса:

- Базовую часть URL, по которому будет выполняться обращение к сервису.
- Состав предоставляемой функциональности и структуру ресурсов, на которых эта функциональность будет отображаться.
- Действия (HTTP-методы), которые можно будет выполнить при обращении к тому или иному ресурсу.
- Для каждого выбранного действия разработчик должен создать специальный метод, написанный на встроенном языке платформы «1С:Предприятие», который и реализует необходимую функциональность. Метод располагается в специальном модуле, связанном с HTTP-сервисом.

После реализации HTTP-сервиса необходимо опубликовать его на веб-сервере с помощью стандартного механизма публикации.

HTTP-сервисы (REST)

Для обращения к HTTP-сервису нужно сформировать URL, который в общем виде выглядит следующим образом: *http://host/base/hs/<корневойURL>/<относительныйURL>*. Рассмотрим более подробно составные части адреса:

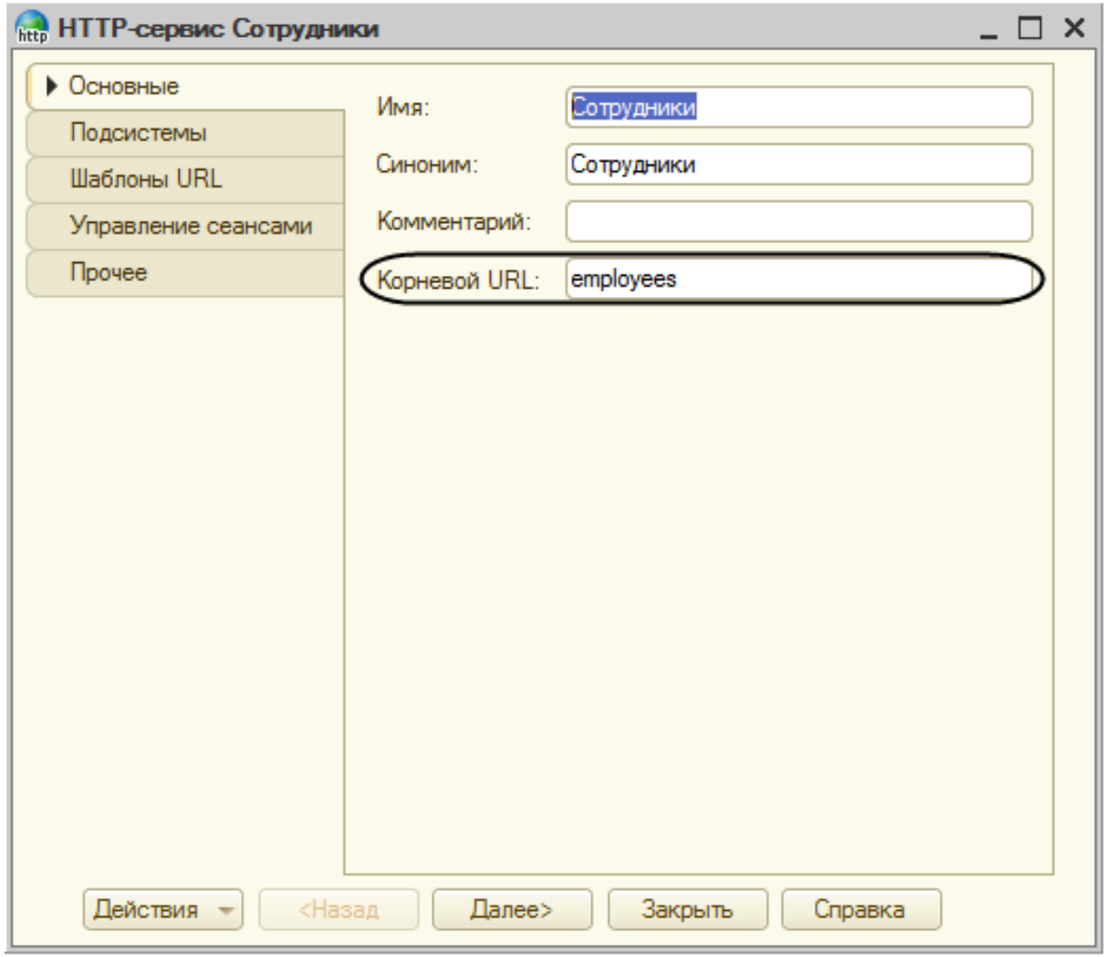
- *http://host/base* – это обычный URL, по которому выполняется доступ, например, к информационной базе с помощью веб-клиента, где base – имя публикуемой базы;
- *hs* – признак того, что выполняется обращение к HTTP-сервису (в отличие от ws, который определяет доступ к Web-сервису);
- *<корневойURL>* – имя ресурса, которое определяет группу ресурсов, связанных общим смыслом. Задается в свойстве КорневойURL объекта HTTP-сервис;
- *<относительныйURL>* – определяет ресурс, к которому будет выполняться обращение. Относительный URL, указанный в запросе, будет использован для определения конкретного ресурса, к которому выполняется обращение. Правило сопоставления задается в свойстве Шаблон объекта Шаблон URL, подчиненного объекту HTTP-сервис.

Разработка HTTP-сервиса

Для создания HTTP-сервисов, которые будет поставлять прикладное решение, есть специальный объект конфигурации HTTP-сервис. Такие объекты добавляются в ветку *Общие > HTTP-сервисы*.

При создании HTTP-сервиса следует установить свойство *Корневой URL*. Оно определяет группу ресурсов, объединенных общим смыслом. Например, если необходимо создать несколько ресурсов, которые работают с заказами, то корневой URL в этом случае может выглядеть как *order*. Тогда начало URL при обращении к такому сервису будет выглядеть следующим образом: *http://host/base/hs/order*.

Примеры реализации HTTP-сервисов



Разработка HTTP-сервиса

Затем в каждом сервисе необходимо создать определенный шаблон URL (группу ресурсов). Таких групп ресурсов может быть несколько. При создании шаблона URL в свойстве Шаблон описываются возможные адреса ресурсов, которые можно будет использовать для обращения к HTTP-сервису. При создании шаблона можно использовать следующие символы:

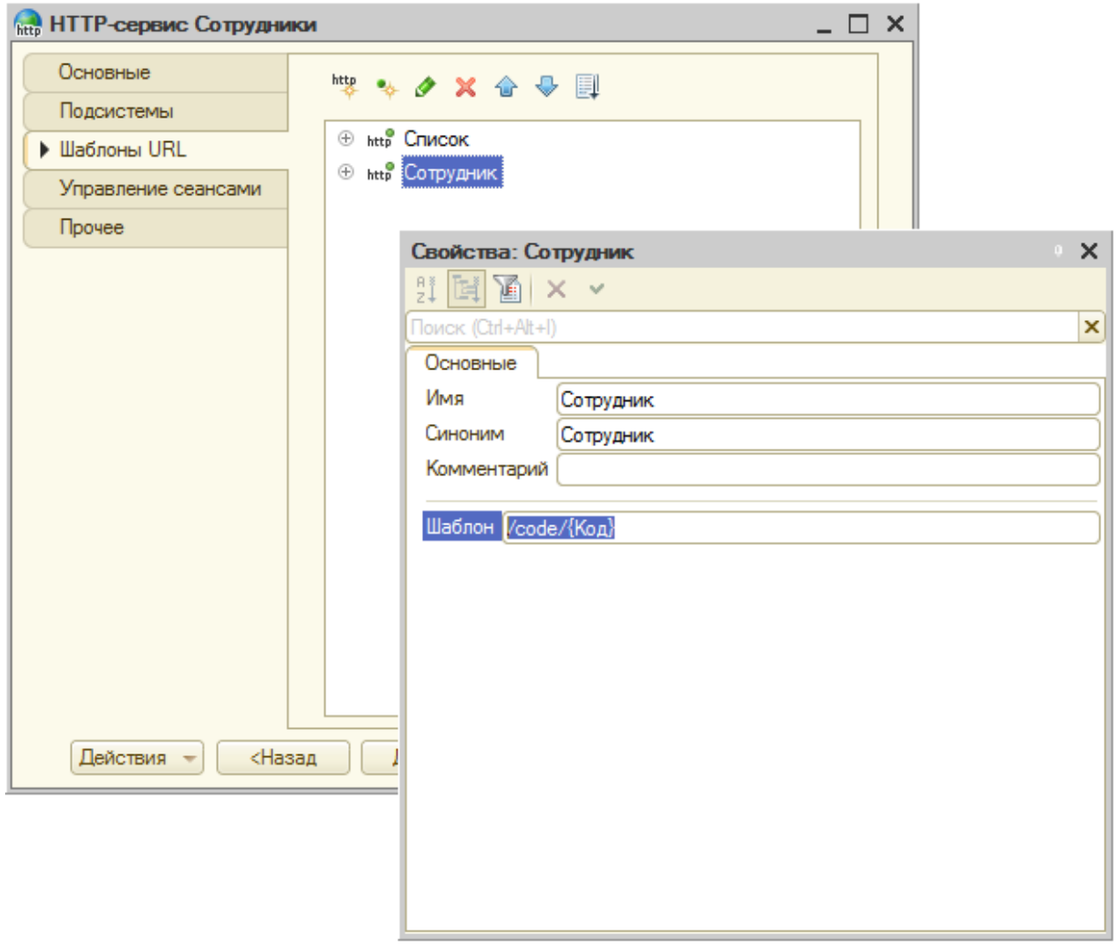
- любые символы, допустимые в идентификаторах языка «1С:Предприятия»;
- символ «/»;
- символы «{}» с непустым текстом между ними;
- символ *.

Например, шаблон может выглядеть следующим образом:

- */query*
- */documents/{id}/props/{PropertyName}/**

В последнем примере *props* – это непараметризованный сегмент, который должен быть перенесен в URL дословно. А заключенный в фигурные скобки *{PropertyName}* – параметризованный сегмент, который фактически описывает переменную с указанным именем, к которой можно получить доступ из встроенного языка.

Примеры реализации HTTP-сервисов



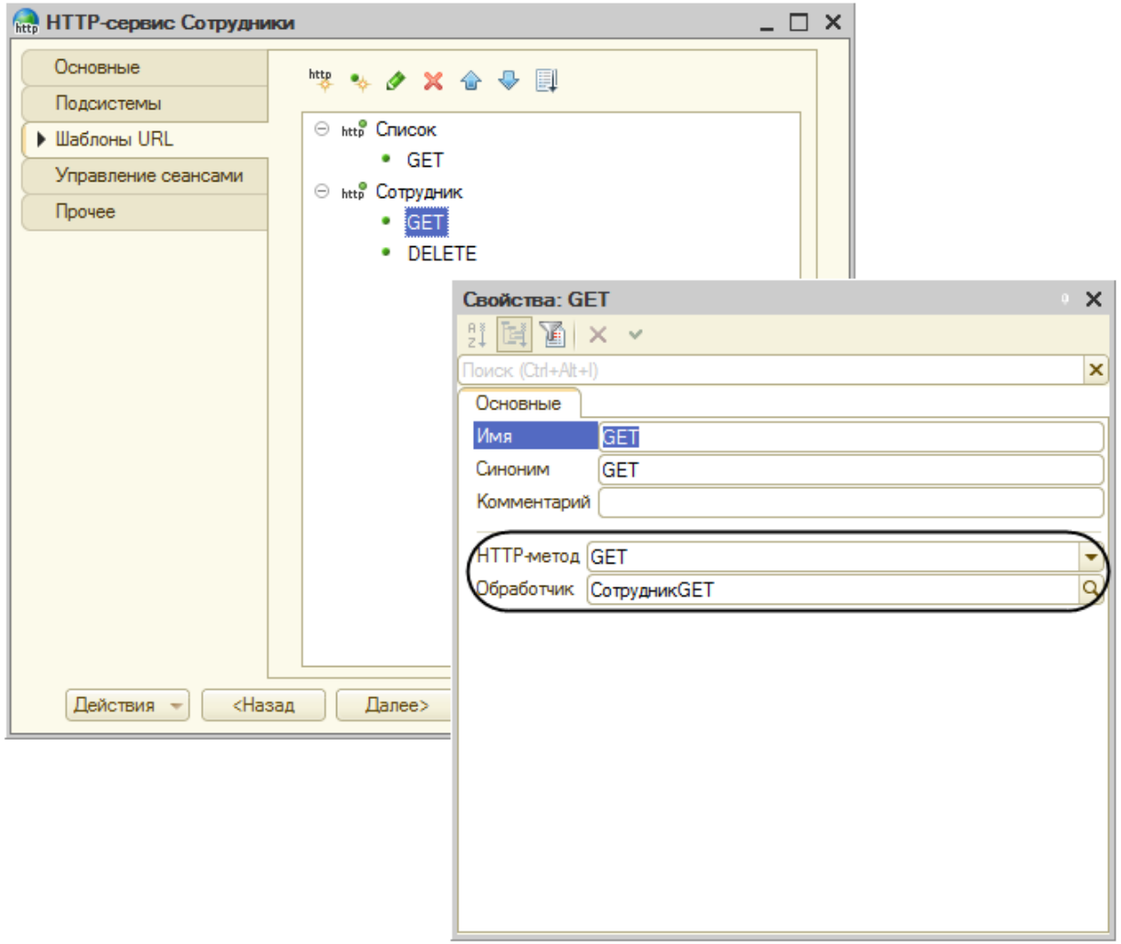
Разработка HTTP-сервиса

После чего, для созданного шаблона необходимо определить HTTP-методы, которые могут быть использованы при работе с ресурсом, совпадающим с шаблоном. Для этого необходимо создать нужное количество объектов *Метод*, подчиненных объекту *Шаблон URL*. Для каждого метода необходимо указать свойство *HTTP-метод* и создать обработчик. В качестве HTTP-метода можно выбрать один из следующих методов: *GET, POST, PUT, DELETE, PATCH, MERGE, CONNECT, OPTIONS, TRACE, PROPFIND, PROPPATCH, MKCOL, COPY, MOVE, LOCK, UNLOCK*. Или можно выбрать значение *Любой* – в этом случае можно будет использовать любой метод из приведенного списка.

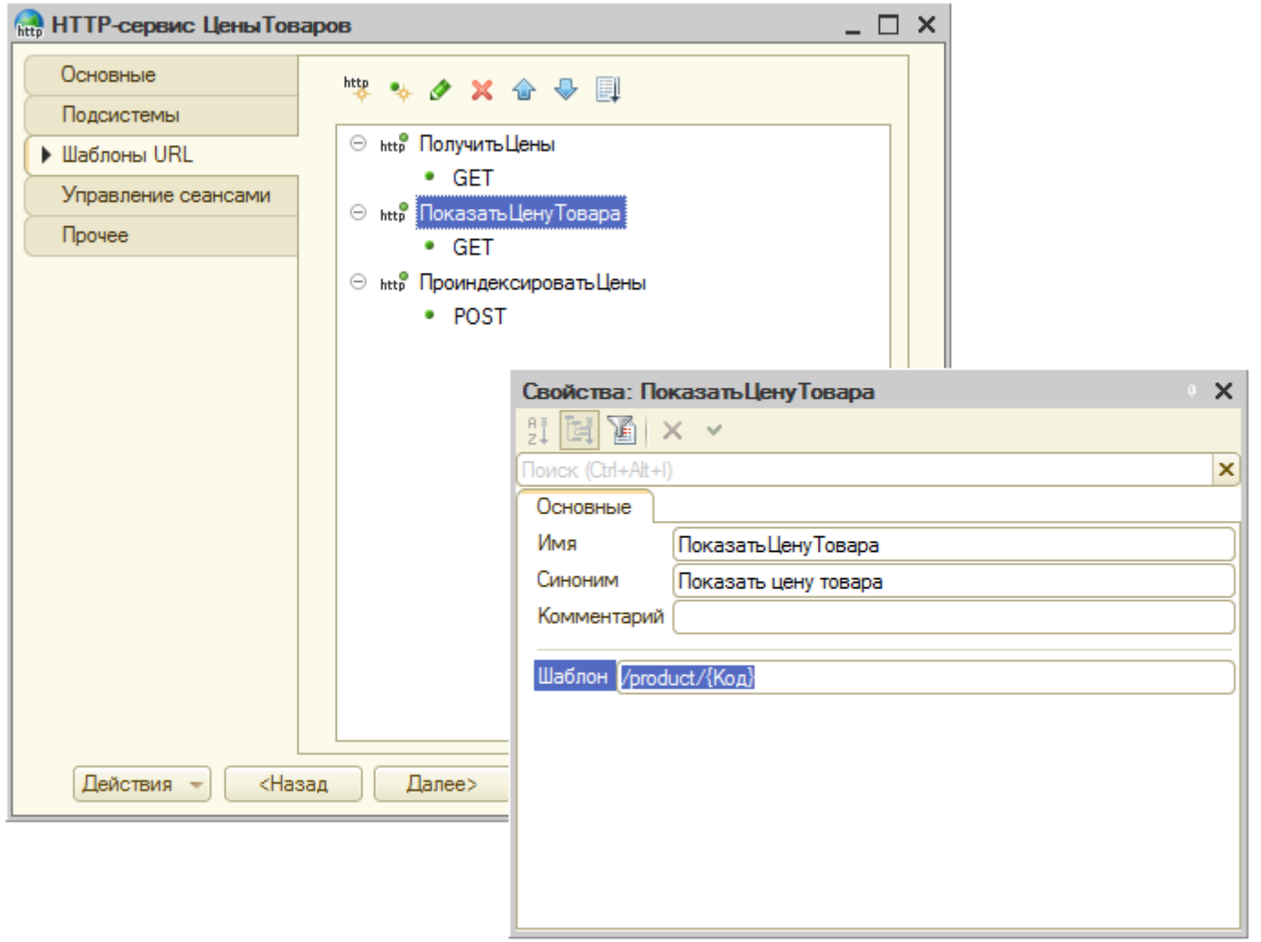
При обращении к HTTP-сервису платформа сначала попытается сопоставить URL, по которому произошло обращение, с одним из имеющихся шаблонов и методов. Если сопоставить не удалось, то платформа выдаст код ответа: 404 – «Not Found». Если подходящий метод будет найден, то платформа начнет выполнение его обработчика.

Обработчиком того или иного HTTP-метода является функция, которая получает на вход один параметр типа *HTTPСервисЗапрос* и должна вернуть объект типа *HTTPСервисОтвет*.

Примеры реализации HTTP-сервисов



Примеры реализации HTTP-сервисов



Разработка HTTP-сервиса

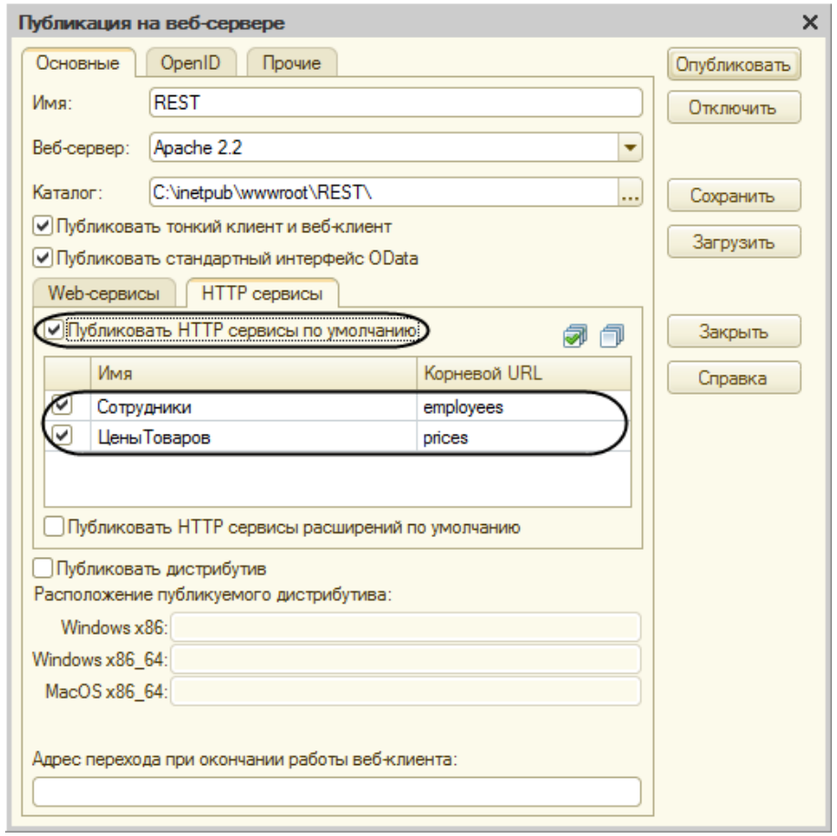
Входной параметр полностью описывает HTTP-запрос, поступивший в систему. С помощью объекта *HTTPСервисЗапрос* можно определить:

- какой HTTP-метод используется в запросе (свойство *HTTPМетод*);
- на какой URL поступил запрос (свойство *БазовыйURL*) – часть полного URL запроса, не включающая относительный URL и параметры запроса;
- относительный URL (свойство *ОтносительныйURL*) – часть полного URL запроса, которая использовалась для определения группы ресурсов и правил обработки входящего HTTP-запроса;
- какие заголовки (свойство *Заголовки*) содержит HTTP-запрос;
- какие параметризованные сегменты (свойство *ПараметрыURL*) выделены из входящего HTTP-запроса и их значения;
- какие параметры (свойство *ПараметрыЗапроса*) перечислены в URL HTTP-запроса после символа "?" и их значения.

После того как прикладное решение выполнило все действия, которые определены для данного ресурса и HTTP-метода, необходимо сформировать ответ сервиса. Для этого необходимо создать объект типа *HTTPСервисОтвет*. В зависимости от результата обработки следует указать свойство *КодСостояния*, которое описывает стандартный код возврата HTTP. Затем, при необходимости, следует сформировать тело ответа HTTP-сервиса. Тело может быть установлено как двоичные данные, как строка (в формате JSON или XML) или с помощью указания имени файла, откуда будет загружено тело ответа.

Разработка HTTP-сервиса

Для работы HTTP-сервиса, его нужно опубликовать на веб-сервере (в качестве веб-сервера могут быть Apache или IIS). Для этого в диалоге публикации информационной базы на веб-сервере на закладке *HTTP сервисы* необходимо установить флажок *Публиковать HTTP сервисы по умолчанию* и отметим имеющиеся HTTP-сервисы, к которым мы хотим предоставить доступ из внешних систем.



Примеры реализации HTTP-сервисов

Чтобы получить список сотрудников с возможным отбором из справочника *Сотрудники*, добавим у нашего HTTP-сервиса объект *Шаблон URL* *Список* с шаблоном /*. Затем у этого объекта добавим подчиненный объект *Метод GET*, в качестве HTTP-метода оставим предложенное по умолчанию значение *GET*. Далее, нажав на кнопку открытия в поле *Обработчик*, создадим обработчик этого метода. Имя обработчика автоматически формируется путем соединения имени шаблона URL и имени самого метода.

```
Функция СписокGET(Запрос)

    // Сформировать ответ, возвращаемый HTTP-сервисом.
    Ответ = Новый HTTPСервисОтвет(200);

    // Получить из запроса параметр URL *.
    Признак = Запрос.ПараметрыURL["*"];

    // Сформировать выборку сотрудников.
    Если Признак = "" Тогда
        // Без отбора.
        Выборка = Справочники.Сотрудники.Выбрать();
    Иначе
        // С отбором по признаку работы.
        Работает = ?(ВРег(Прав(Признак,2)) = "ДА", Истина,
        Ложь);
        Выборка = Справочники.Сотрудники.Выбрать(,, Новый С
        труктура("Работает", Работает));
    КонецЕсли;

    // В цикле обхода выборки записать в JSON список
    сотрудников.
    // Создать объект записи и записать строковое значение в
    строку JSON.
    Запись = Новый ЗаписьJSON;
    Запись.УстановитьСтроку();
    // Записать начало корневого объекта.
    Запись.ЗаписатьНачалоОбъекта();
    Пока Выборка.Следующий() Цикл
        Запись.ЗаписатьИмяСвойства(Выборка.Наименование);
        Запись.ЗаписатьНачалоОбъекта();
        Запись.ЗаписатьИмяСвойства("Код");
        Запись.ЗаписатьЗначение(Выборка.Код);
        Запись.ЗаписатьИмяСвойства("Должность");
        Запись.ЗаписатьЗначение(Строка(Выборка.Должность));
        Запись.ЗаписатьКонецОбъекта();
    КонецЦикла;
    // Записать конец корневого объекта.
    Запись.ЗаписатьКонецОбъекта();

    // Записать результат записи в строку JSON.
    Результат = Запись.Закрыть();

    // Установить тело ответа из строки Результат.
    Ответ.УстановитьТелоИзСтроки(Результат);
    Ответ.Заголовки.Вставить("Content-
    type", "application/json");

    Возврат Ответ;

КонецФункции
```

Примеры реализации HTTP-сервисов

В переменной *Ответ* мы сразу же создаем объект типа *HTTPСервисОтвет*, который содержит ответ, формируемый HTTP-сервером на поступивший запрос. Ответ имеет стандартный код возврата HTTP (*КодСостояния*=200). Код состояния 200 сообщает об успешном выполнении запроса. Поскольку в функции мы описываем простейшее действие – получение выборки данных из справочника, то для простоты мы не будем здесь анализировать ошибочные состояния, которые могут возникнуть при выполнении этого обработчика.

Родительский объект функции-обработчика – это шаблон URL Список, который содержит строку шаблона /*. Этот шаблон означает, что в относительном URL на месте звездочки могут оказаться любые допустимые символы, в том числе возможно и отсутствие этих символов.

Поэтому мы получаем и анализируем те символы, которые в реальном URL присутствуют на месте звездочки. Для этого мы получаем параметры URL запроса, который передается в функцию в параметре *Запрос* с помощью конструкции *Запрос.ПараметрыURL["*"]*.

Если относительный URL пустой, то мы формируем выборку из справочника *Сотрудники* без отбора. Если на месте звездочки находится строка «ДА», то она интерпретируется как *Истина*, любые другие символы интерпретируется как Ложь. Соответственно этому признаку накладывается отбор на поле справочника *Работает*.

В цикле обхода выборки мы заполняем строку JSON списком сотрудников и затем записываем эту строку в переменную Результат. После этого с помощью метода УстановитьТелоИзСтроки() устанавливаем тело ответа HTTP-сервиса из строки Результат и возвращаем ответ сервиса в переменной Ответ.

При этом заголовок «Content-type» объекта HTTPСервисОтвет мы устанавливаем как «application/json». Такой заголовок помогает клиенту понять, что же за данные к нему пришли, кроме того, из него определяется кодировка ответа, если она не задана явно.

Примеры реализации HTTP-сервисов

```
http://localhost/REST/hs/employees
```



```
{
"Алексеев Сергей Иванович": {
"Код": "000000001",
"Должность": "Администратор"
},
"Артемов Игорь Владимирович": {
"Код": "REST-0003",
"Должность": "Новая должность - REST"
},
"Смирнова Светлана Ивановна": {
"Код": "000000002",
"Должность": "Кассир"
}
}
```

```
http://localhost/REST/hs/employees/да
```



```
{
"Алексеев Сергей Иванович": {
"Код": "000000001",
"Должность": "Администратор"
},
"Артемов Игорь Владимирович": {
"Код": "REST-0003",
"Должность": "Новая должность - REST"
}
}
```

Примеры реализации HTTP-сервисов

Чтобы получить данные о конкретном сотруднике, необходимо воспользоваться другим шаблоном URL HTTP-сервиса. В относительном URL запроса при обращении к сервису должно быть указано некое уникальное свойство (например, *Код*), по которому сотрудника можно было бы идентифицировать в справочнике. В обработчике, связанном с этим шаблоном, данные о конкретном сотруднике должны быть записаны в строку JSON и подставлены в тело ответа, возвращаемого сервисом.

Итак, добавим в HTTP-сервис объект *Шаблон URL Сотрудник* с шаблоном */code/{Код}*. Затем у этого объекта добавим подчиненный метод *GET*, в качестве HTTP-метода оставим предложенное по умолчанию значение *GET*. Далее создадим обработчик этого метода.

В модуле HTTP-сервиса будет создан шаблон функции-обработчика *СотрудникGET()*.

Примеры реализации HTTP-сервисов

```
Функция СотрудникGET(Запрос)

    // Сформировать ответ, возвращаемый HTTP-сервисом.
    Ответ = Новый HTTPСервисОтвет(200);

    // Получить из запроса параметр URL Код.
    Код = Запрос.ПараметрыURL.Получить("Код");
    Если Код = Неопределено Тогда
        Ответ = Новый HTTPСервисОтвет(400);
        Ответ.УстановитьТелоИзСтроки("Не задан параметр
        Код");
        Ответ.Заголовки.Вставить("Content-
        type", "application/json");
        Возврат Ответ;
    КонецЕсли;

    // Найти сотрудника в справочнике по полученному
    параметру Код.
    СотрудникСсылка = Справочники.Сотрудники.НайтиПоКод
    у(Код);
    Если СотрудникСсылка = Справочники.Сотрудники.Пуста
    яСсылка() Тогда
        Ответ = Новый HTTPСервисОтвет(404);
        Ответ.УстановитьТелоИзСтроки("Сотрудник не
        найден");
        Ответ.Заголовки.Вставить("Content-
        type", "application/json");
        Возврат Ответ;
    КонецЕсли;

    Сотрудник = СотрудникСсылка.ПолучитьОбъект();

    // Сериализовать данные объекта Сотрудник с помощью
    объекта записи (Запись).
    ПараметрыЗаписиJSON = Новый ПараметрыЗаписиJSON(Пер
    еносСтрокJSON.Авто, Символы.Таб);
    Запись = Новый ЗаписьJSON;
    Запись.УстановитьСтроку(ПараметрыЗаписиJSON);
    СериализаторXDTO.ЗаписатьJSON(Запись, Сотрудник);

    // Записать результат записи в строку JSON.
    Результат = Запись.Закрыть();

    // Установить тело ответа из строки Результат.
    Ответ.УстановитьТелоИзСтроки(Результат);
    Ответ.Заголовки.Вставить("Content-
    type", "application/json");

    Возврат Ответ;

КонецФункции
```

Примеры реализации HTTP-сервисов

```
Функция СотрудникDELETE(Запрос)

    // Сформировать ответ (без тела),
    возвращаемый HTTP-сервисом.
    Ответ = Новый HTTPСервисОтвет(204);

    // Получить из запроса параметр URL Код.
    Код = Запрос.ПараметрыURL.Получить("Код")
    ;
    Если Код = Неопределено Тогда
        Ответ = Новый HTTPСервисОтвет(400);
        Ответ.УстановитьТелоИзСтроки("Не задан
        параметр Код");
        Ответ.Заголовки.Вставить("Content-
        type", "application/json");
        Возврат Ответ;
    КонецЕсли;

    // Найти сотрудника в справочнике по
    полученному параметру Код.
    СотрудникСсылка = Справочники.Сотрудники.
    НайтиПоКоду(Код);

    Если СотрудникСсылка = Справочники.Сотруд
    ники.ПустаяСсылка() Тогда
        Ответ = Новый HTTPСервисОтвет(404);
        Ответ.УстановитьТелоИзСтроки("Сотрудни
        к не найден");
        Ответ.Заголовки.Вставить("Content-
        type", "application/json");
        Возврат Ответ;
    КонецЕсли;

    // Установить найденному сотруднику
    пометку удаления.
    Сотрудник = СотрудникСсылка.ПолучитьОбъек
    т();
    Сотрудник.УстановитьПометкуУдаления(Истин
    а);
    Сотрудник.Записать();

    Возврат Ответ;

КонецФункции
```

Примеры реализации HTTP-сервисов

В следующем примере рассмотрим, как с помощью HTTP-сервиса получать информацию о ценах товаров, которые в нашей демонстрационной конфигурации хранятся в регистре сведений *ЦеныТоваров*.

Для этого в ветке *Общие* > *HTTP-сервисы* добавим HTTP-сервис с именем *ЦеныТоваров*, свойство которого *Корневой URL* установим как *prices*.

Поскольку в дальнейшем HTTP-сервис будет опубликован на локальном сервере в каталоге «REST», то базовая часть URL для всех шаблонов URL (групп ресурсов) при обращении к сервису будет выглядеть как *http://localhost/REST/hs/prices*.

Далее, рассмотрим примеры по работе с ценами в Регистре сведений. Сперва получим цены товаров на определенную дату:

```
Функция ПолучитьЦеныGET(Запрос)

    Попытка

        // Получить из запроса параметр URL Дата.
        ДатаЗапроса = Запрос.ПараметрыURL.Получить("Дата");
        Если ДатаЗапроса = Неопределено Тогда
            Ответ = Новый HTTPСервисОтвет(400);
            Ответ.УстановитьТелоИзСтроки("Не задан параметр Дата");
            Ответ.Заголовки.Вставить("Content-
            type", "application/json");
            Возврат Ответ;
        КонецЕсли;

        ЦеныНаДату = Дата(ДатаЗапроса);
        // Сформировать запрос к регистру сведений.
        Запрос = Новый Запрос;
        Запрос.Текст = "ВЫБРАТЬ
        |    ЦеныТоваровСрезПоследних.Период КАК Период,
        |    ЦеныТоваровСрезПоследних.Товар КАК Товар,
        |    ЦеныТоваровСрезПоследних.Цена КАК Цена
        |ИЗ
        |    РегистрСведений.ЦеныТоваров.СрезПоследних(&ЦеныНаДату, )
        КАК ЦеныТоваровСрезПоследних";

        Запрос.УстановитьПараметр("ЦеныНаДату", ЦеныНаДату);
        Выборка = Запрос.Выполнить().Выбрать();

        // В цикле обхода выборки записать в JSON последние цены
        товаров
        // Создать объект записи и записать строковое значение в
        строку JSON.
        Запись = Новый ЗаписьJSON;
        Запись.УстановитьСтроку();

        // Записать начало корневого объекта.
        Запись.ЗаписатьНачалоОбъекта();
        Пока Выборка.Следующий() Цикл
            Запись.ЗаписатьИмяСвойства(Строка(Выборка.Товар));
            Запись.ЗаписатьНачалоОбъекта();
            Запись.ЗаписатьИмяСвойства("Период");
            Запись.ЗаписатьЗначение(Строка(Выборка.Период));
            Запись.ЗаписатьИмяСвойства("Цена");
            Запись.ЗаписатьЗначение(Выборка.Цена);
            Запись.ЗаписатьКонецОбъекта();
        КонецЦикла;
        // Записать конец корневого объекта.
        Запись.ЗаписатьКонецОбъекта();

        // Записать результат записи в строку JSON.
        Результат = Запись.Закрыть();

        // Сформировать ответ, возвращаемый HTTP-сервисом.
        Ответ = Новый HTTPСервисОтвет(200);

        // Установить тело ответа из строки Результат.
        Ответ.УстановитьТелоИзСтроки(Результат);

    Исключение

        // Вывести стуктурированную информацию об исключении.
        Ответ = Новый HTTPСервисОтвет(500);
        Информация = ИнформацияОбОшибке();
        Сообщение = Информация.Описание;
        Если Информация.Причина <> Неопределено Тогда
                Сообщение = Сообщение + ":" + Информация.Причина.Опи
        сание;
        КонецЕсли;
        Ответ.УстановитьТелоИзСтроки(Сообщение);

    КонецПопытки;

    Ответ.Заголовки.Вставить("Content-type", "application/json");
    Возврат Ответ;

КонецФункции
```

Примеры реализации HTTP-сервисов

```
http://localhost/REST/hs/prices/date/20190510
```




```
{
"Чайник": {
"Период": "07.05.2019 0:00:00",
"Цена": 3000
},
"Тостер": {
"Период": "07.05.2019 0:00:00",
"Цена": 2000
},
"Кофемашина": {
"Период": "07.05.2019 0:00:00",
"Цена": 5000
}
}
```

Теперь получим цену конкретного товара на определенную дату:

```
Функция ПоказатьЦенуТовараGET(Запрос)

Попытка

	// Получить из запроса параметр URL Код.
	Код = Запрос.ПараметрыURL.Получить("Код");
	Если Код = Неопределено Тогда
		Ответ = Новый HTTPСервисОтвет(400);
		Ответ.УстановитьТелоИзСтроки("Не задан параметр Код");
		Ответ.Заголовки.Вставить("Content-
		type", "application/json");
		Возврат Ответ;
	КонецЕсли;

	// Найти товар в справочнике по полученному параметру Код.
	Товар = Справочники.Товары.НайтиПоКоду(Код);
	Если Товар = Справочники.Товары.ПустаяСсылка() Тогда
		Ответ = Новый HTTPСервисОтвет(400);
		Ответ.УстановитьТелоИзСтроки("Товар не найден");
		Ответ.Заголовки.Вставить("Content-
		type", "application/json");
		Возврат Ответ;
	КонецЕсли;

	// Получить последнюю цену товара из регистра сведений.
	ЦенаТовара = РегистрыСведений.ЦеныТоваров.ПолучитьПоследнее(, Но
	вый Структура("Товар", Товар));

	// Создать объект записи и записать строковое значение в строку
	JSON.
	Запись = Новый ЗаписьJSON;
	Запись.УстановитьСтроку();

	// Записать начало корневого объекта.
	Запись.ЗаписатьНачалоОбъекта();
	Запись.ЗаписатьИмяСвойства("На: ");
	Запись.ЗаписатьЗначение(Строка(ТекущаяДата()));
	Запись.ЗаписатьИмяСвойства("Товар");
	Запись.ЗаписатьЗначение(Строка(Товар));
	Запись.ЗаписатьИмяСвойства("Цена");
	Запись.ЗаписатьЗначение(ЦенаТовара.Цена);
	// Записать конец корневого объекта.
	Запись.ЗаписатьКонецОбъекта();

	// Записать результат записи в строку JSON.
	Результат = Запись.Закрыть();

	// Сформировать ответ, возвращаемый HTTP-сервисом.
	Ответ = Новый HTTPСервисОтвет(200);

	// Установить тело ответа из строки Результат.
	Ответ.УстановитьТелоИзСтроки(Результат);

Исключение

	// Вывести стуктурированную информацию об исключении.
	Ответ = Новый HTTPСервисОтвет(500);
	Информация = ИнформацияОбОшибке();
	Сообщение = Информация.Описание;
	Если Информация.Причина <> Неопределено Тогда
		Сообщение = Сообщение + ":" + Информация.Причина.Описание;
	КонецЕсли;
	Ответ.УстановитьТелоИзСтроки(Сообщение);

КонецПопытки;

Ответ.Заголовки.Вставить("Content-type", "application/json");
Возврат Ответ;

КонецФункции
```

Примеры реализации HTTP-сервисов

```
http://localhost/REST/hs/prices/product/000000001
```



```
{
"На: ": "21.07.2019 13:06:01",
"Товар": "Чайник",
"Цена": 3300
}
```

Далее проиндексируем цены на товар:

```
Функция ПроиндексироватьЦеныPOST(Запрос)

Попытка

// Получить из запроса параметр URL Процент.
Процент = Запрос.ПараметрыURL.Получить("Процент");
Если Процент = Неопределено Тогда
      Ответ = Новый HTTPСервисОтвет(400);
      Ответ.УстановитьТелоИзСтроки("Не задан параметр Процент");
      Ответ.Заголовки.Вставить("Content-type", "application/json");
      Возврат Ответ;
КонецЕсли;

Коэффициент = 1 + Число(Процент)/100;
// Сформировать запрос к регистру сведений.
Запрос = Новый Запрос;
Запрос.Текст = "ВЫБРАТЬ
|    ЦеныТоваровСрезПоследних.Период КАК Период,
|    ЦеныТоваровСрезПоследних.Товар КАК Товар,
|    ЕСТЬNULL(ЦеныТоваровСрезПоследних.Цена, 0) КАК Цена
|ИЗ
|    РегистрСведений.ЦеныТоваров.СрезПоследних(, ) КАК
ЦеныТоваровСрезПоследних";

Выборка = Запрос.Выполнить().Выбрать();

// В цикле обхода выборки записать в JSON и добавить в регистр сведений
последние цены товаров.
// Создать объект записи и записать строковое значение в строку JSON.
Запись = Новый ЗаписьJSON;
Запись.УстановитьСтроку();

// Записать начало корневого объекта.
Запись.ЗаписатьНачалоОбъекта();
Пока Выборка.Следующий() Цикл
      НоваяЦена = Выборка.Цена * Коэффициент;

      Запись.ЗаписатьИмяСвойства(Строка(Выборка.Товар));
      Запись.ЗаписатьНачалоОбъекта();
      Запись.ЗаписатьИмяСвойства("Период");
      Запись.ЗаписатьЗначение(Строка(ТекущаяДата()));
      Запись.ЗаписатьИмяСвойства("Цена");
      Запись.ЗаписатьЗначение(НоваяЦена);
      Запись.ЗаписатьКонецОбъекта();

      // Подготовить менеджер записи.
      ЗаписьРегистра = РегистрыСведений.ЦеныТоваров.СоздатьМенеджерЗаписи();

      // Установить ключевые поля менеджера записи.
      ЗаписьРегистра.Период = ТекущаяДата();
      ЗаписьРегистра.Товар = Выборка.Товар;
      ЗаписьРегистра.Цена = НоваяЦена;

      // Записать новую запись в базу данных.
      ЗаписьРегистра.Записать();
КонецЦикла;
// Записать конец корневого объекта.
Запись.ЗаписатьКонецОбъекта();

// Записать результат записи в строку JSON.
Результат = Запись.Закрыть();

// Сформировать ответ, возвращаемый HTTP-сервисом.
Ответ = Новый HTTPСервисОтвет(200);

// Установить тело ответа из строки Результат.
Ответ.УстановитьТелоИзСтроки(Результат);

Исключение

// Вывести стуктурированную информацию об исключении.
Ответ = Новый HTTPСервисОтвет(500);
Информация = ИнформацияОбОшибке();
Сообщение = Информация.Описание;
Если Информация.Причина <> Неопределено Тогда
            Сообщение = Сообщение + ":" + Информация.Причина.Описание;
КонецЕсли;
Ответ.УстановитьТелоИзСтроки(Сообщение);

КонецПопытки;

Ответ.Заголовки.Вставить("Content-type", "application/json");
Возврат Ответ;

КонецФункции
```

HTTP-запросы

Из системы «1С:Предприятие» можно обратиться к любому произвольному HTTP-сервису, опубликованному в Интернете.

Для обращения к HTTP-сервисам и используются такие объекты встроенного языка, как HTTPСоединение, HTTPЗапрос и HTTPОтвет.

- Объект *HTTPСоединение* предназначен для взаимодействия с внешними системами по протоколу HTTP. В том числе с помощью объекта *HTTPСоединение* можно:
  - создать HTTP-соединение;
  - записать, получить и удалить файл;
  - отправить ресурс на указанный адрес для обработки;
  - определить параметры установленного соединения;
  - вызвать произвольный HTTP-метод;
  - передать на сервер заголовки запроса и получить заголовки с сервера;
  - в качестве тела запроса отправить не только файл, но и строку (с автоматическим перекодированием в нужную кодировку), и двоичные данные.
- Объект *HTTPЗапрос* предназначен для описания HTTP-запросов, отправляемых через объект *HTTPСоединение*. Позволяет задать адрес запрашиваемого ресурса, заголовки и тело запроса.
- Объект *HTTPОтвет* – предоставляет доступ к содержимому ответа HTTP-сервера на запрос.

HTTP-запросы

Для того чтобы изменить (добавить, удалить, прочитать и т. д.) данные информационной базы через REST-интерфейс, нужно создать HTTP-соединение с сервером (например, "localhost"), на котором опубликованы HTTP-сервисы (как созданные разработчиком, так и составляющие автоматический REST-интерфейс) для прикладного решения. Затем создать HTTP-запрос на основе адреса ресурса данных, которые требуется прочитать или изменить с помощью запроса. После этого нужно записать изменяемые данные в строку JSON/XML (или в файл, откуда будет считано тело запроса при отправке) и установить тело HTTP-запроса из этой строки (или установить имя файла тела запроса). И затем с помощью HTTP-соединения отправить запрос к серверу с соответствующим методом (*POST, PUT/PATCH, DELETE* и др.).
Для обращения к HTTP-сервисам и используются такие объекты встроенного языка, как HTTPСоединение, HTTPЗапрос и HTTPОтвет.

Чтобы выполнить HTTP-запрос нужного вида, у объекта *HTTPСоединение* существуют методы:

- *Получить()* – отправляет GET-запрос на сервер;
- *ОтправитьДляОбработки()* – отправляет POST-запрос на сервер;
- *Изменить()* – отправляет PATCH-запрос на сервер;
- *Записать()* – отправляет PUT-запрос на сервер;
- *Удалить()* – отправляет DELETE-запрос на сервер;
- с помощью метода *ВызватьHTTPМетод()* можно отправить на сервер произвольный запрос, выполняющий HTTP-метод, имя которого указано в первом параметре метода *ВызватьHTTPМетод()*.

Обращение к HTTP-сервисам

```
&НаКлиенте
Процедура ПолучитьДанные(Команда)
	// Сформировать строку URL.
	АдресРесурса = СокрЛП(Адрес);
	// Отправить запрос на сервер.
	ВыполнитьЗапрос("GET", АдресРесурса, "");
КонецПроцедуры

&НаКлиенте
Процедура ВыполнитьЗапрос(ИмяМетода, АдресРесурса, СтрокаТелаЗапроса)

Сообщение = Новый СообщениеПользователю;
// Установить имя сервера.
СерверИсточник = "localhost";

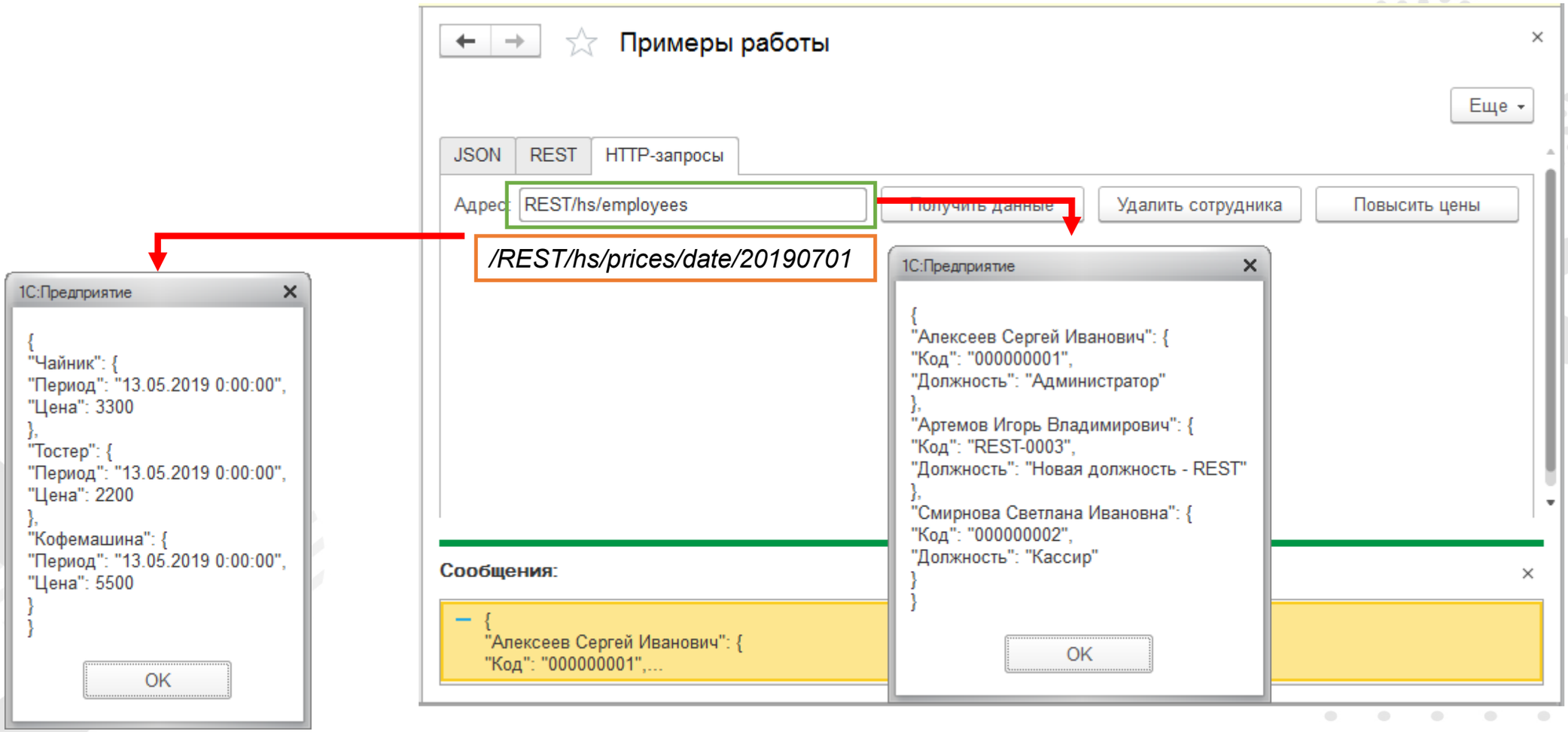
Попытка
	// Создать HTTP-соединение с сервером localhost.
	HTTPСоединение = Новый HTTPСоединение(СерверИсточник);
Исключение
	// Вывести сообщение об ошибке соединения с сервером.
	Сообщение.Текст = "Не удалось соединиться с сервером:
	" + СерверИсточник;
	Сообщение.Сообщить();
	Сообщение.Текст = ОписаниеОшибки();
	Сообщение.Сообщить();
	Возврат;
КонецПопытки;

// Создать HTTP-запрос на основе URL.
HTTPЗапрос = Новый HTTPЗапрос(АдресРесурса);
// Установить тело запроса из строки JSON.
Если ИмяМетода <> "DELETE" И ИмяМетода <> "GET" Тогда
	HTTPЗапрос.УстановитьТелоИзСтроки(СтрокаТелаЗапроса);
КонецЕсли;

Попытка
	// Получить ответ сервера в виде объекта HTTPОтвет.
	Результат = HTTPСоединение.ВызватьHTTPМетод(ИмяМетода, HTTP
	Запрос);
	// Получить содержимое ответа сервера в виде строки.
	Сообщение.Текст = Результат.ПолучитьТелоКакСтроку();
	Сообщение.Сообщить();
Исключение
	// Вывести сообщение об ошибке при получении ответа
	сервера.
	Сообщение.Текст = ОписаниеОшибки();
	Сообщение.Сообщить();
	Возврат;
КонецПопытки;

КонецПроцедуры
```

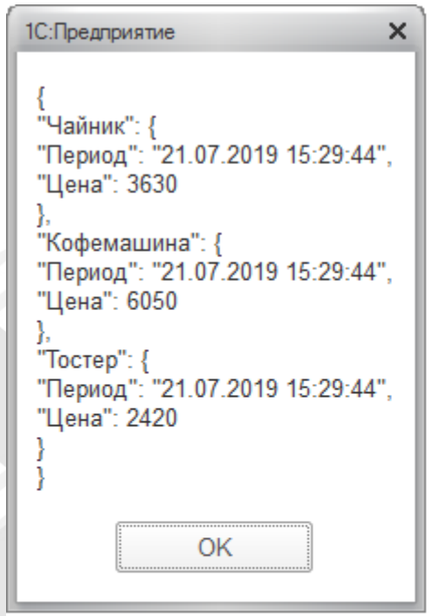
Обращение к HTTP-сервисам

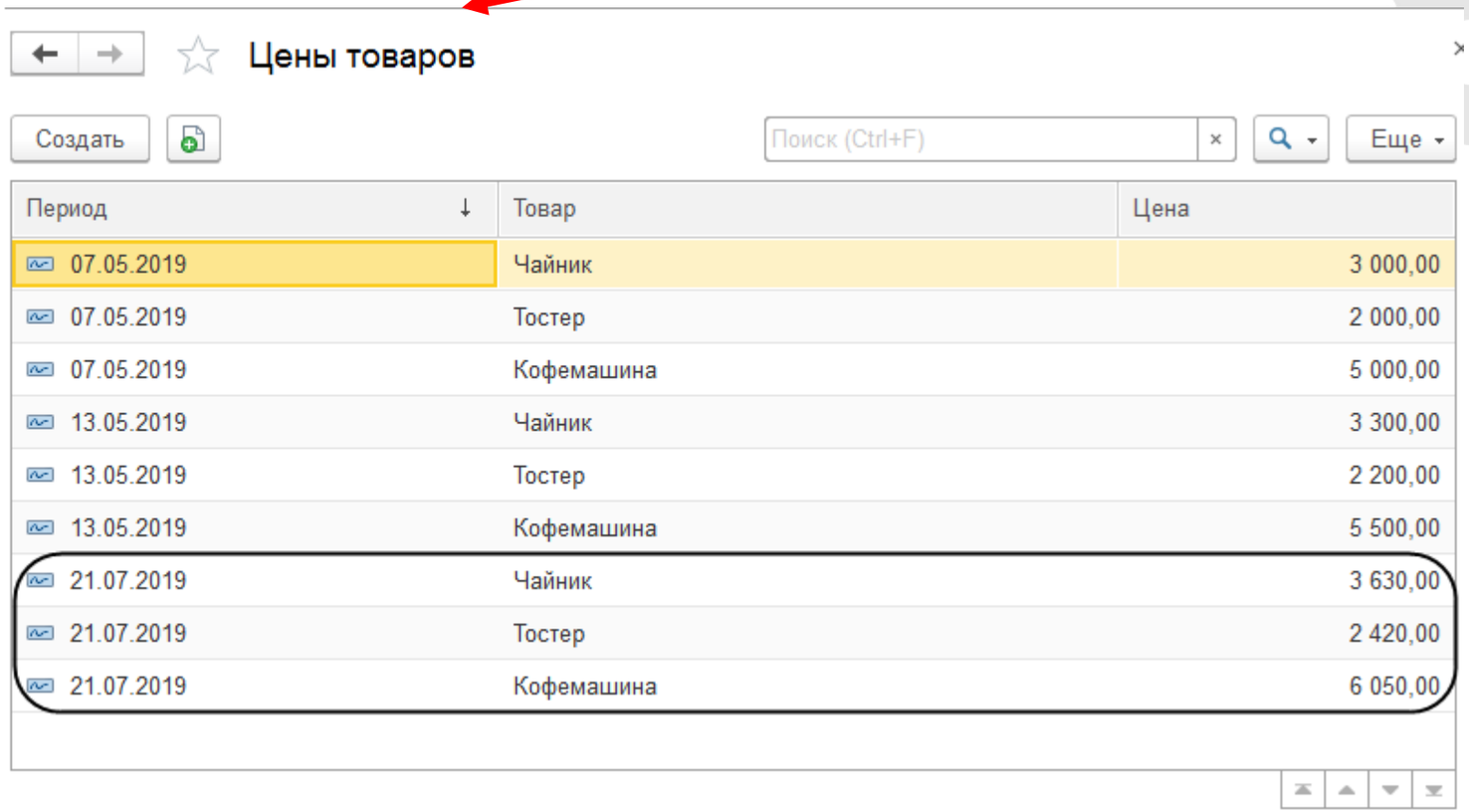


Обращение к HTTP-сервисам

```
&НаКлиенте
Процедура УдалитьСотрудника(Команда)
      // Сформировать строку URL.
      АдресРесурса = СокрЛП(Адрес);
      // Отправить запрос на сервер.
      ВыполнитьЗапрос("DELETE", АдресРесурса, "");
КонецПроцедуры
```

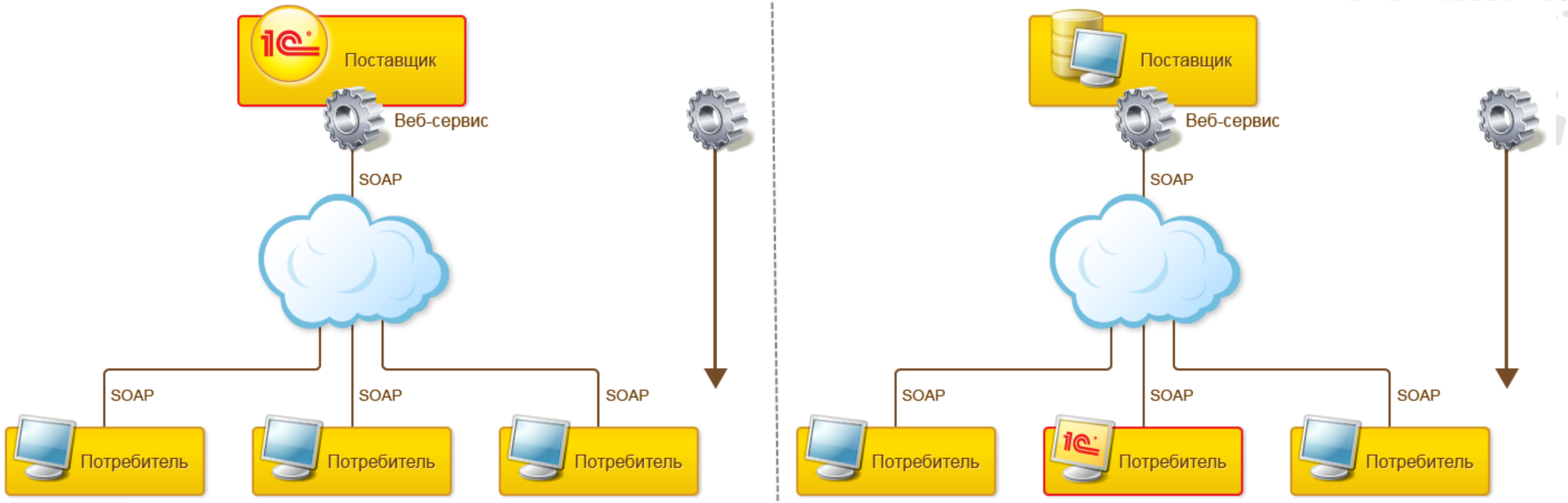
```
&НаКлиенте
Процедура ПроиндексироватьЦены(Команда)
      // Сформировать строку URL.
      АдресРесурса = СокрЛП(Адрес);
      // Отправить запрос на сервер.
      ВыполнитьЗапрос("POST", АдресРесурса, "");
КонецПроцедуры
```





Web-сервисы

Прикладное решение «1С:Предприятия» может являться как поставщиком веб-сервисов, так и потребителем веб-сервисов, опубликованных другими поставщиками.



Web-сервисы

Для получения доступа к Web-сервису необходимо использовать адрес, который формируется следующим образом: http://host/base/ws/<ИмяWebСервиса> или http://host/base/ws/<АдресWebСервиса>. Рассмотрим составные части адреса:

- *http://host/base* – обычный URL, по которому выполняется доступ, например, к информационной базе с помощью веб-клиента. Необходимо помнить, что при использовании информационной базы, в которой настроено разделение данных, значения разделителей можно указывать только в URL информационной базы. Указание разделителей с помощью параметра Z не поддерживается;
- *ws* – признак того, что выполняется обращение к Web-сервису;
- *<ИмяWebСервиса>* – имя Web-сервиса. Задается в свойстве объекта Web-сервис;
- *<АдресWebСервиса>* – описывает альтернативное имя для доступа к Web-сервису. Задается в свойстве Имя файла публикации объекта Web-сервис. Может быть изменено при публикации Web-сервиса.

Разработка Web-сервиса

Для создания Web-сервиса нужно:

- Добавить в дерево конфигурации объект WebСервис и описать основные свойства этого объекта:
  - *URI пространства имен*. Каждый Web-сервис может быть однозначно идентифицирован по своему имени и URI пространства имен, которому он принадлежит;
  - *Пакеты XDTO*. Содержит перечень пакетов XDTO, типы которых могут использоваться для описания типов параметров и возвращаемых значений Web-сервиса;
  - *Имя файла публикации*. Содержит имя файла описания Web-сервиса, который опубликован на веб-сервере.
- Описать операции, которые может выполнять данный Web-сервис. Основные свойства операций:
  - *Тип возвращаемого значения*. Содержит тип значения, которое возвращает операция Web-сервиса. Может являться типом значения XDTO или типом объекта XDTO;
  - *Возможно пустое значение*. Показывает, может ли значение, возвращаемое операцией, принимать неопределенное значение;
  - *Имя метода*. Содержит имя процедуры, расположенной в модуле Web-сервиса, которая будет выполняться при вызове данного свойства.
- Описать параметры выполняемых операций. Основные свойства параметров:
  - *Тип значения*. Содержит тип значения параметра операции Web-сервиса. Может являться типом значения XDTO или типом объекта XDTO;
  - *Возможно пустое значение*. Показывает, может ли параметр операции принимать неопределенное значение;
  - *Направление передачи*. Определяет направление передачи данных с помощью параметра. Входной параметр используется для передачи данных Web-сервису, Выходной – для получения данных от Web-сервиса, Входной-выходной может использоваться как для передачи данных, так и для их получения от Web-сервиса.
- Создать модуль Web-сервиса и разработать в нем операции, определяющие его функциональность.

Объект конфигурации WebСервис содержит модуль, в котором создаются процедуры на встроенном языке, выполняемые при вызове тех или иных операций Web-сервиса. Типы параметров и возвращаемых значений этих операций описываются с помощью типов XDTO и могут представлять собой либо значения XDTO, либо объекты XDTO.

Механизм XDTO

Механизм XDTO – это один из механизмов интеграции с другими системами. Аббревиатура XDTO расшифровывается как XML Data Transfer Objects. XDTO является механизмом объектного моделирования данных, описываемых с помощью схемы XML.

Основные возможности использования XDTO:

- описание типов параметров и возвращаемых значений Web-сервисов;
- обмен данными между конфигурациями 1С:Предприятия 8 с существенно разными структурами данных;
- обмен данными на основе схем XML, не привязанных к той или иной конфигурации (например, обмен с информационными системами, построенными не на основе 1С:Предприятия 8);
- создание собственной системы типов и значений для обработки произвольных данных.

XDTO пакет

Механизм XDTO реализован с помощью набора объектов встроенного языка и объекта конфигурации XDTO-пакет. Они позволяют описать систему типов и значений, которая будет использоваться для взаимодействия с другими программными системами.

XDTO-сериализация

В системе 1С:Предприятие 8 реализована сериализация данных на основе механизма XDTO, которая позволяет сериализовать в/из XML:

- все типы данных, хранящиеся в базе данных
- некоторые другие типы

XDTO-сериализация предназначена для сохранения данных объекта в файл XML и создания объекта на основе данных, хранящихся в файле XML.

Работа с Web-сервисами сторонних поставщиков

Система «1С:Предприятие» может использовать Web-сервисы, предоставляемые другими поставщиками, следующими способами:

- с помощью статических ссылок, создаваемых в дереве объектов конфигурации;
- с помощью динамических ссылок, создаваемых средствами встроенного языка;
- комбинацией предыдущих способов.

При использовании статической ссылки система «1С:Предприятие» получает описание Web-сервиса поставщика только один раз, при создании ссылки. За счет этого достигается большая скорость работы.

При использовании динамической ссылки описание Web-сервиса получается каждый раз при вызове Web-сервиса. Скорость работы при этом уменьшается, но зато такой подход обеспечивает актуальность описания Web-сервиса поставщика. В случае же использования статических ссылок для получения актуального описания Web-сервиса требуется выполнить повторный импорт WSDL-описания средствами конфигуратора и затем сохранить измененную конфигурацию.

Работа с Web-сервисами сторонних поставщиков

Например, для Web-сервиса с URI пространства имен *http://www.MyCompany.ru/shipment*, именем *ДанныеРасходнойНакладной* и именем файла публикации *Shipment.1cws* статическая ссылка создается путем импорта WSDL-описания опубликованного сервиса на основе URL – *http://www.MyCompany.ru/shipment/ws/Shipment.1cws?wsdl*. А создание WS-прокси на основании WS-ссылки будет выглядеть следующим образом.

```
// Создать WS-прокси на основании WS-ссылки.
Прокси = WSСсылки.ДанныеРасходнойНакладной.СоздатьWSПрокси("http://www.MyCompany.ru/shipment"
, "ДанныеРасходнойНакладной", "ДанныеРасходнойНакладнойSoap");
```

При использовании динамической ссылки на этот Web-сервис WS-прокси создается на основании WS-определения:

```
// Создать WS-прокси на основании WS-определения.
Определение = Новый WSОпределения("http://www.MyCompany.ru/shipment/ws/Shipment.1cws?wsdl");
Прокси = Новый WSПрокси(Определение, "http://www.MyCompany.ru/shipment", "ДанныеРасходнойНакл
адной", "ДанныеРасходнойНакладнойSoap");
```

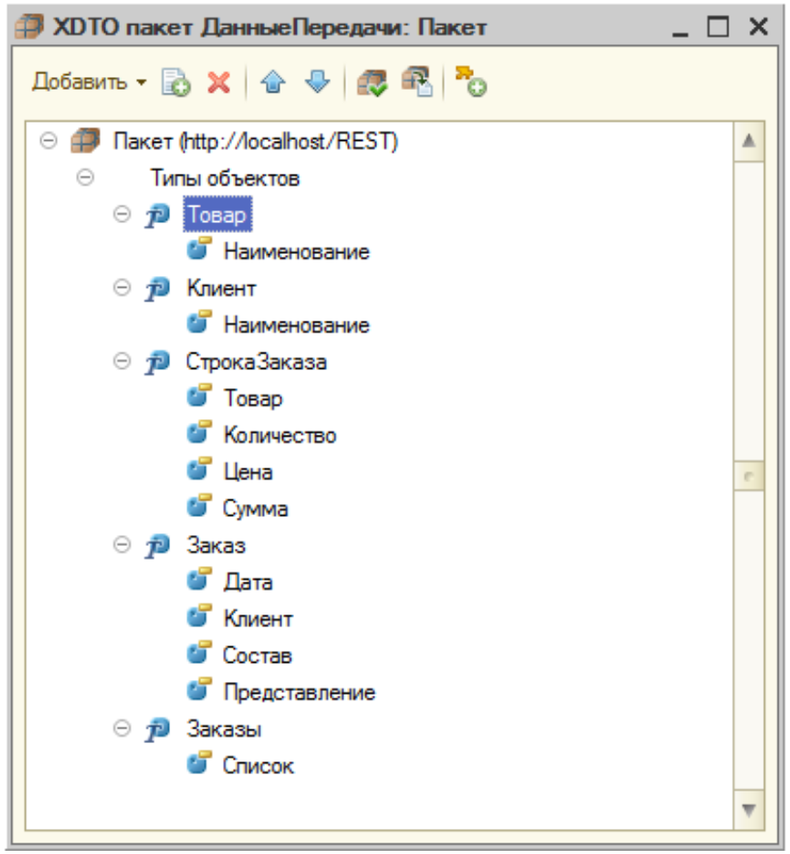
Пример реализации Web-сервиса

Рассмотрим ситуацию, когда в одном прикладном решении существуют заказы товаров, а в другом прикладном решении на основании этих заказов, не дожидаясь сеанса обмена данными, должны быть созданы расходные накладные. При этом нормативно-справочная информация у этих решений общая.

Для этого в прикладном решении поставщика Web-сервиса нам нужно разработать Web-сервис, который будет получать незакрытые заказы товаров и возвращать всю информацию о таких заказах. И затем опубликовать этот сервис на веб-сервере. А в прикладном решении потребителя Web-сервиса нам нужно реализовать функциональность (например, команду), при выполнении которой на основании полученных от Web-сервиса заказов будут сформированы соответствующие документы продаж.

Пример реализации Web-сервиса

Сначала для описания типов параметров и возвращаемых значений Web-сервиса в ветке *Общие* > *XDTO-пакеты* добавим пакет XDTO *ДанныеПередачи* с пространством имен *http://localhost/REST*. В свойстве *URI пространства имен* содержится *http://localhost* – адрес веб-сервера, установленного на локальном компьютере, */REST* – каталог, в который будет опубликован Web-сервис.



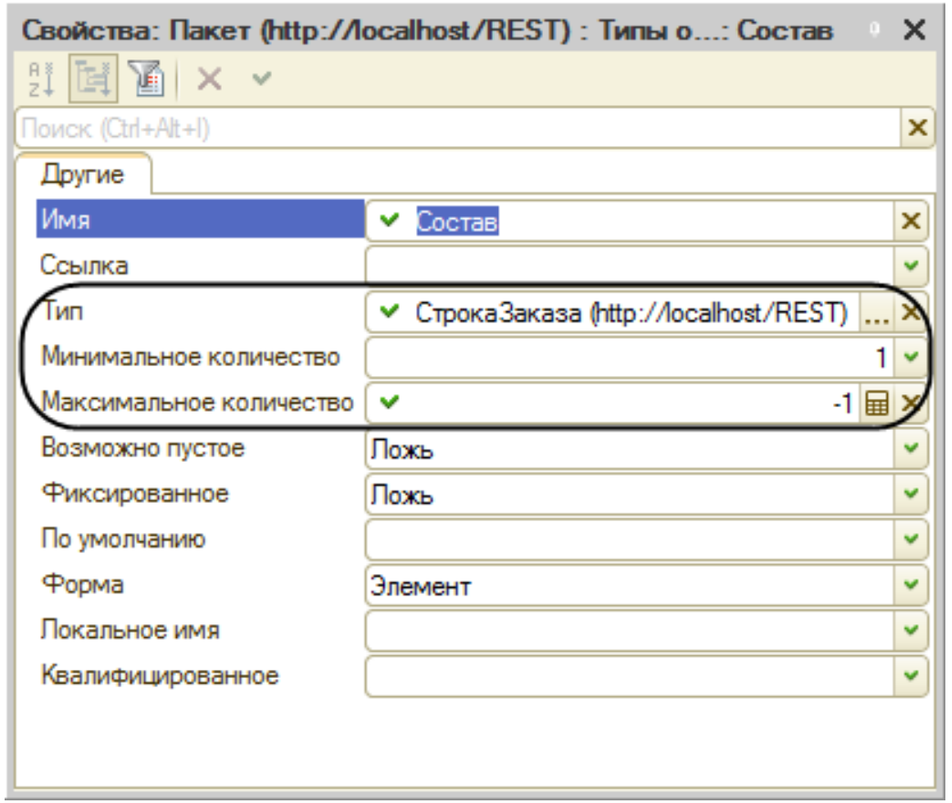
Пример реализации Web-сервиса

Пакет XDTO будет содержать следующие типы объектов XDTO:

- *Товар* – для передачи данных элемента справочника *Товары*. Этот тип объектов XDTO будет содержать свойство:
  - *Наименование* – тип *string* из пространства имен http://www.w3.org/2001/XMLSchema.
- *Клиент* – для передачи данных элемента справочника *Клиенты*. Этот тип объектов XDTO будет содержать свойство:
  - *Наименование* – тип *string* из пространства имен http://www.w3.org/2001/XMLSchema.
- *СтрокаЗаказа* – для передачи данных одной строки состава заказа товаров. Этот тип объектов XDTO будет содержать следующие свойства:
  - *Товар* – тип *Товар* из пространства имен http://localhost/REST. Представляет собой ссылку на объект XDTO, определенный нами ранее, в предыдущем примере;
  - *Количество* – тип *int* из пространства имен http://www.w3.org/2001/XMLSchema;
  - *Цена* – тип *int* из пространства имен http://www.w3.org/2001/XMLSchema;
  - *Сумма* – тип *int* из пространства имен http://www.w3.org/2001/XMLSchema.
- *Заказ* – для передачи всех данных одного заказа. Этот тип объектов XDTO будет содержать следующие свойства:
  - *Дата* – тип *date* из пространства имен http://www.w3.org/2001/XMLSchema;
  - *Клиент* – тип *Клиент* из пространства имен http://localhost/REST. Представляет собой ссылку на объект XDTO, определенный нами выше;
  - *Состав* – тип *СтрокаЗаказа* из пространства имен http://localhost/REST. Представляет собой ссылку на объект XDTO, определенный нами выше. Для того чтобы это свойство могло содержать неограниченное множество значений, установим его свойство *Максимальное количество* в значение -1;
  - *Представление* – тип *string* из пространства имен http://www.w3.org/2001/XMLSchema.
- *Заказы* – для передачи данных всех заказов. Этот тип объектов XDTO будет содержать единственное свойство:
  - *Список* – тип *Заказ* из пространства имен http://localhost/REST. Представляет собой ссылку на объект XDTO, определенный нами выше. Для того чтобы это свойство могло содержать неограниченное множество значений, установим его свойство Максимальное количество в значение -1.

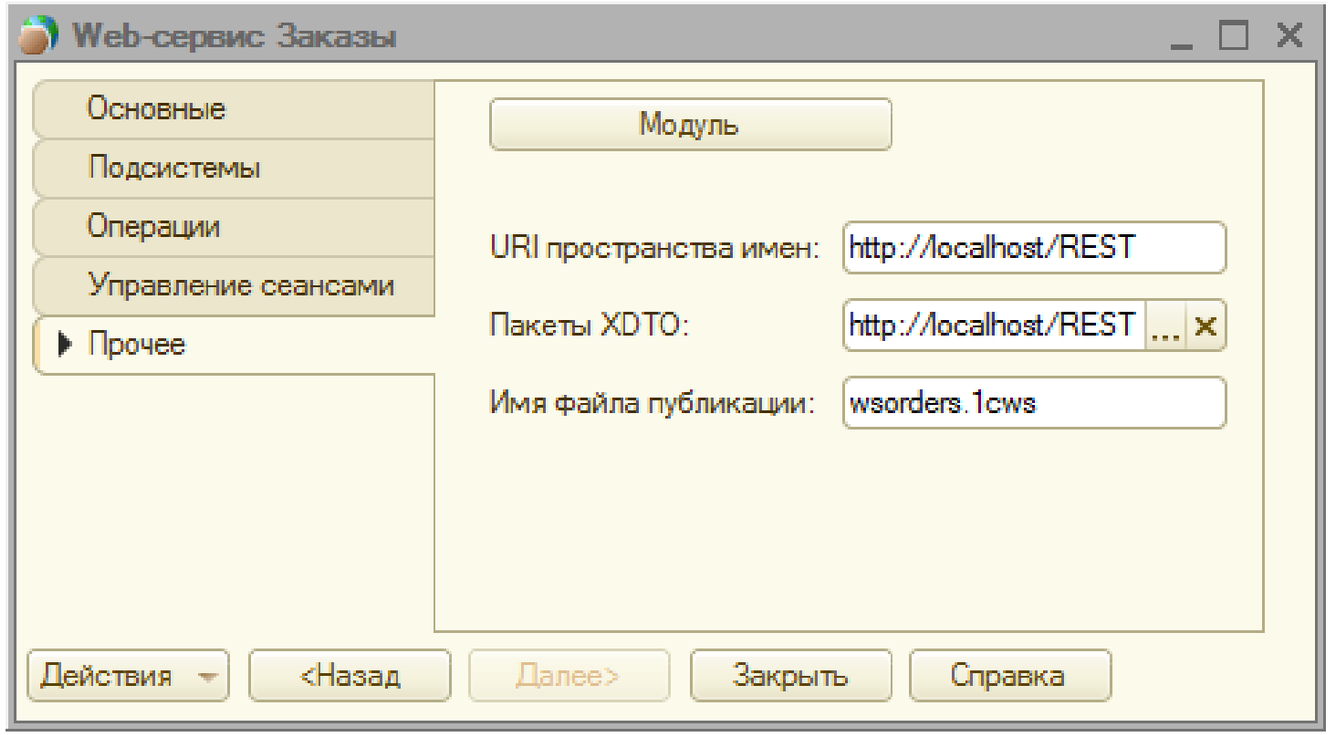
Пример реализации Web-сервиса

Свойство состав определим следующим образом:



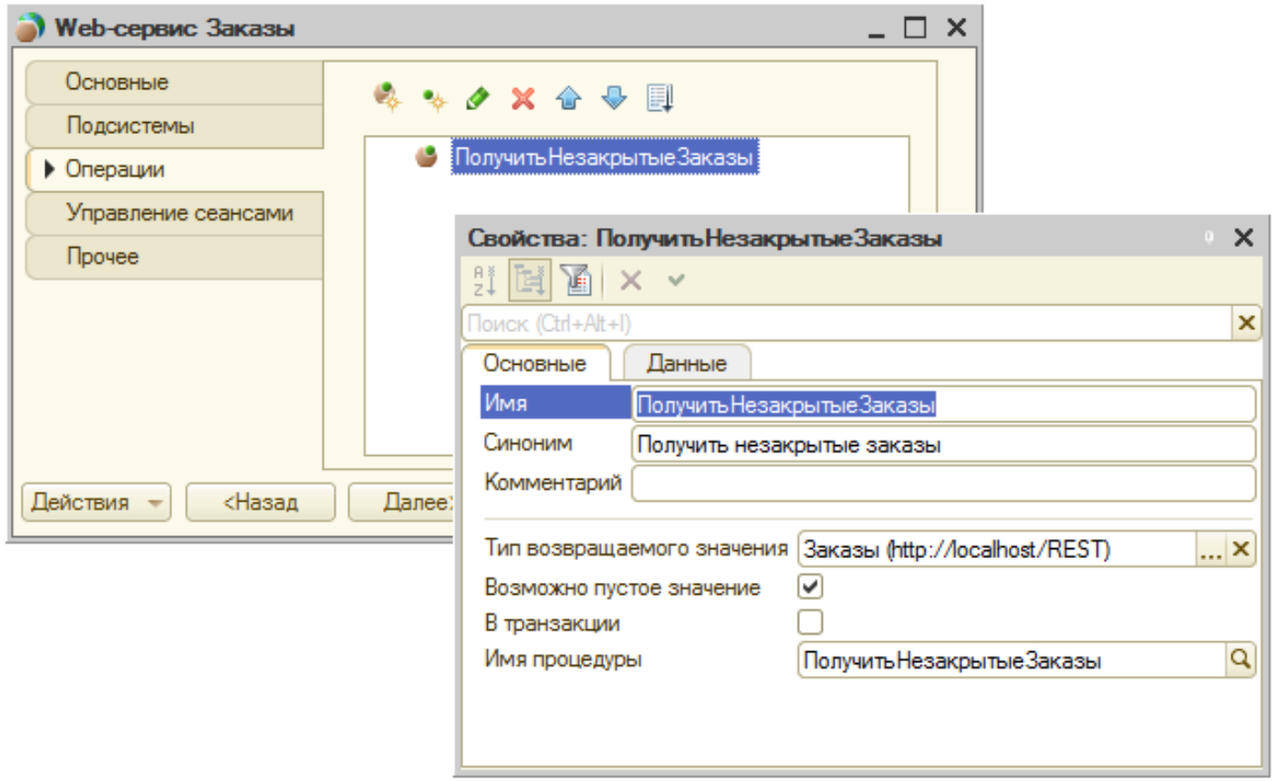
Пример реализации Web-сервиса

После того, как необходимые типы объектов XDTO созданы, в ветке *Общие* > *Web-сервисы* добавим Web-сервис с именем *Заказы* со следующими свойствами:



Пример реализации Web-сервиса

На закладке *Операции* в окне свойств Web-сервиса определим операцию *ПолучитьНезакрытыеЗаказы* со следующими свойствами:



Эта операция будет возвращать список всех незакрытых заказов на товары, т. е. тех заказов, у которых признак закрытия еще не установлен и, следовательно, на их основании еще не создавались расходные накладные.

Нажмем на кнопку открытия со значком лупы у свойства *Имя процедуры*. В результате в модуле Web-сервиса будет создан шаблон функции-обработчика *ПолучитьНезакрытыеЗаказы()*, которая будет выполняться при его вызове. Заполним эту функцию следующим образом.

```
Функция ПолучитьНезакрытыеЗаказы()

// Сформировать запрос к списку незакрытых заказов.
Запрос = Новый Запрос;
Запрос.Текст = "ВЫБРАТЬ
|    Заказ.Ссылка КАК Ссылка,
|    Заказ.Дата КАК Дата,
|    Заказ.Клиент КАК Клиент,
|    Заказ.Товары.(
|         Товар КАК Товар,
|         Количество КАК Количество,
|         Цена КАК Цена,
|         Сумма КАК Сумма
|    ) КАК Товары,
|    Заказ.Представление КАК ПредставлениеЗаказа
|ИЗ
|    Документ.Заказ КАК Заказ
|ГДЕ
|    Заказ.Закрыт = ЛОЖЬ
|    И Заказ.Проведен = ИСТИНА";

РезультатЗапроса = Запрос.Выполнить();
Если РезультатЗапроса.Пустой() Тогда
Возврат Неопределено;
КонецЕсли;

Выборка = РезультатЗапроса.Выбрать();

// Получить типы объектов XDTO.
ТоварТип = ФабрикаXDTO.Тип("http://localhost/REST", "Товар");
КлиентТип = ФабрикаXDTO.Тип("http://localhost/REST", "Клиент");
СтрокаЗаказаТип = ФабрикаXDTO.Тип("http://localhost/REST", "СтрокаЗ
аказа");
ЗаказТип = ФабрикаXDTO.Тип("http://localhost/REST", "Заказ");
ЗаказыТип = ФабрикаXDTO.Тип("http://localhost/REST", "Заказы");

// Создать объект XDTO список заказов.
Заказы = ФабрикаXDTO.Создать(ЗаказыТип);

// В цикле обхода выборки получить список и содержимое незакрытых
заказов.
Пока Выборка.Следующий() Цикл
// Создать объект XDTO Заказ и Клиент.
Заказ = ФабрикаXDTO.Создать(ЗаказТип);
Клиент = ФабрикаXDTO.Создать(КлиентТип);
// Заполнить свойства клиента и заказа.
Клиент.Наименование = Выборка.Клиент.Наименование;
Заказ.Клиент = Клиент;
Заказ.Дата = Выборка.Дата;
Заказ.Представление = Выборка.ПредставлениеЗаказа;

ВыборкаСостав = Выборка.Товары.Выбрать();
// В цикле обхода состава выборки получить состав заказа
Пока ВыборкаСостав.Следующий() Цикл
// Создать объекты XDTO строки заказа и товара.
СтрокаЗаказа = ФабрикаXDTO.Создать(СтрокаЗаказаТип);
Товар = ФабрикаXDTO.Создать(ТоварТип);
// Заполнить свойства товара и строки заказа.
Товар.Наименование = ВыборкаСостав.Товар.Наименование;
СтрокаЗаказа.Товар = Товар;
СтрокаЗаказа.Цена = ВыборкаСостав.Цена;
СтрокаЗаказа.Количество  = ВыборкаСостав.Количество;
СтрокаЗаказа.Сумма = ВыборкаСостав.Сумма;
// Добавить строку заказа.
Заказ.Состав.Добавить(СтрокаЗаказа);
КонецЦикла;

// Добавить заказ в список заказов.
Заказы.Список.Добавить(Заказ);

// Закрыть заказ.
Документ = Выборка.Ссылка.ПолучитьОбъект();
Документ.Закрыт = Истина;
Документ.Записать();

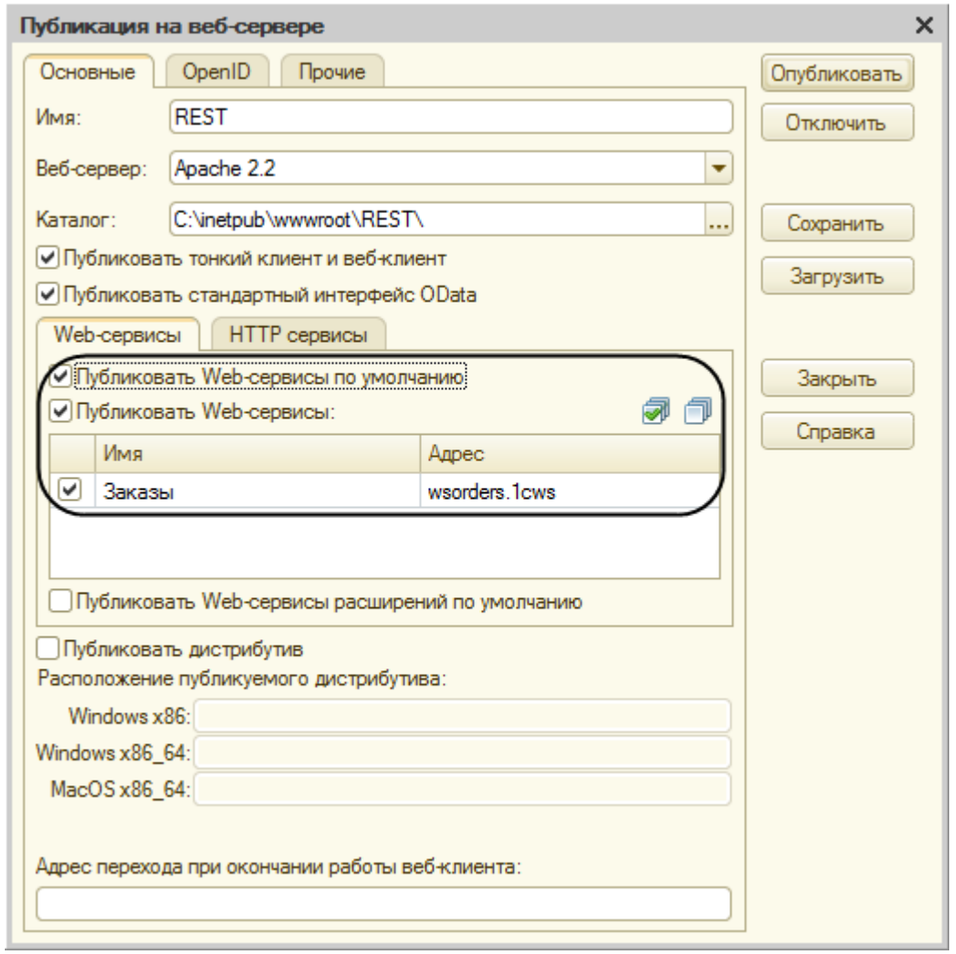
КонецЦикла;

Возврат Заказы;

КонецФункции
```

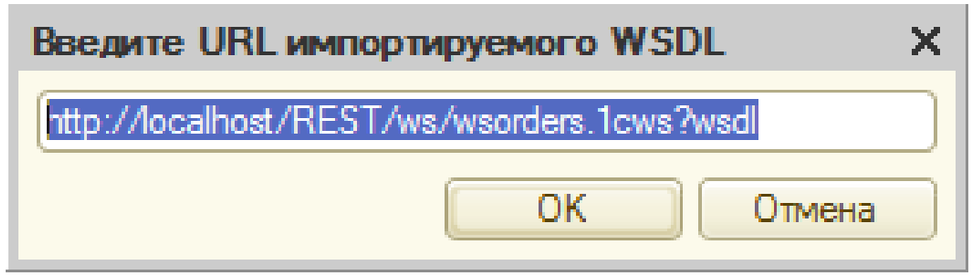
Пример реализации Web-сервиса

Далее необходимо опубликовать созданный Web-сервис на веб-сервере – например, расположенном на локальном компьютере http://localhost в каталоге /REST.



Пример реализации Web-сервиса

Чтобы использовать функциональность, предоставляемую Web-сервисом *Заказы*, в прикладном решении «1С:Предприятия», необхолимо в дереве объектов конфигурации в ветку *Общие* > *WS-ссылки* добавить объект *WSCсылка* с именем *ДанныеЗаказов*, ссылающийся на опубликованный Web-сервис. При создании ссылки мы выполнили импорт WSDL-описания Web-сервиса, указав в качестве URL http://localhost/REST/ws/wsorders.1cws?wsdl. И реализовать серверную процедуру *ЗакрытьЗаказыНаСервере*() и команду с кнопкой, например, на форме обработки.



Пример реализации Web-сервиса

```
&НаСервереБезКонтекста
Процедура ЗакрытьЗаказыНаСервере()

// Создать WS-прокси на основании WS-ссылки.
Прокси = WSСсылки.ДанныеЗаказов.СоздатьWSПрокси("
http://localhost/REST", "Заказы", "ЗаказыSoap");

// Выполнить операцию Веб-сервиса Заказы.
СписокЗаказов = Прокси.ПолучитьНезакрытыеЗаказы()
;
Если СписокЗаказов = Неопределено Тогда
    Возврат;
КонецЕсли;

Для Каждого Заказ Из СписокЗаказов.Список Цикл
// Создать расходную накладную на основании
заказа.
ДокументОбъект = Документы.РасходнаяНакладная.Соз
датьДокумент();
ДокументОбъект.Дата = Заказ.Дата;
// Найти клиента по наименованию.
ДокументОбъект.Клиент = Справочники.Клиенты.Найти
ПоНаименованию(Заказ.Клиент.Наименование);
ДокументОбъект.Основание = Заказ.Представление;

// Заполнить ТЧ расходной накладной данными
заказа.
Для Каждого СтрокаСостава Из Заказ.Состав Цикл
НоваяСтрока = ДокументОбъект.Товары.Добавить();
НоваяСтрока.Количество = СтрокаСостава.Количество
;
НоваяСтрока.Цена = СтрокаСостава.Цена;
НоваяСтрока.Сумма = СтрокаСостава.Сумма;

// Найти товар по наименованию.
НоваяСтрока.Товар = Справочники.Товары.НайтиПоНаи
менованию(СтрокаСостава.Товар.Наименование);
КонецЦикла;

ДокументОбъект.Записать(, РежимПроведенияДокумент
а.Неоперативный);
КонецЦикла;

КонецПроцедуры
```

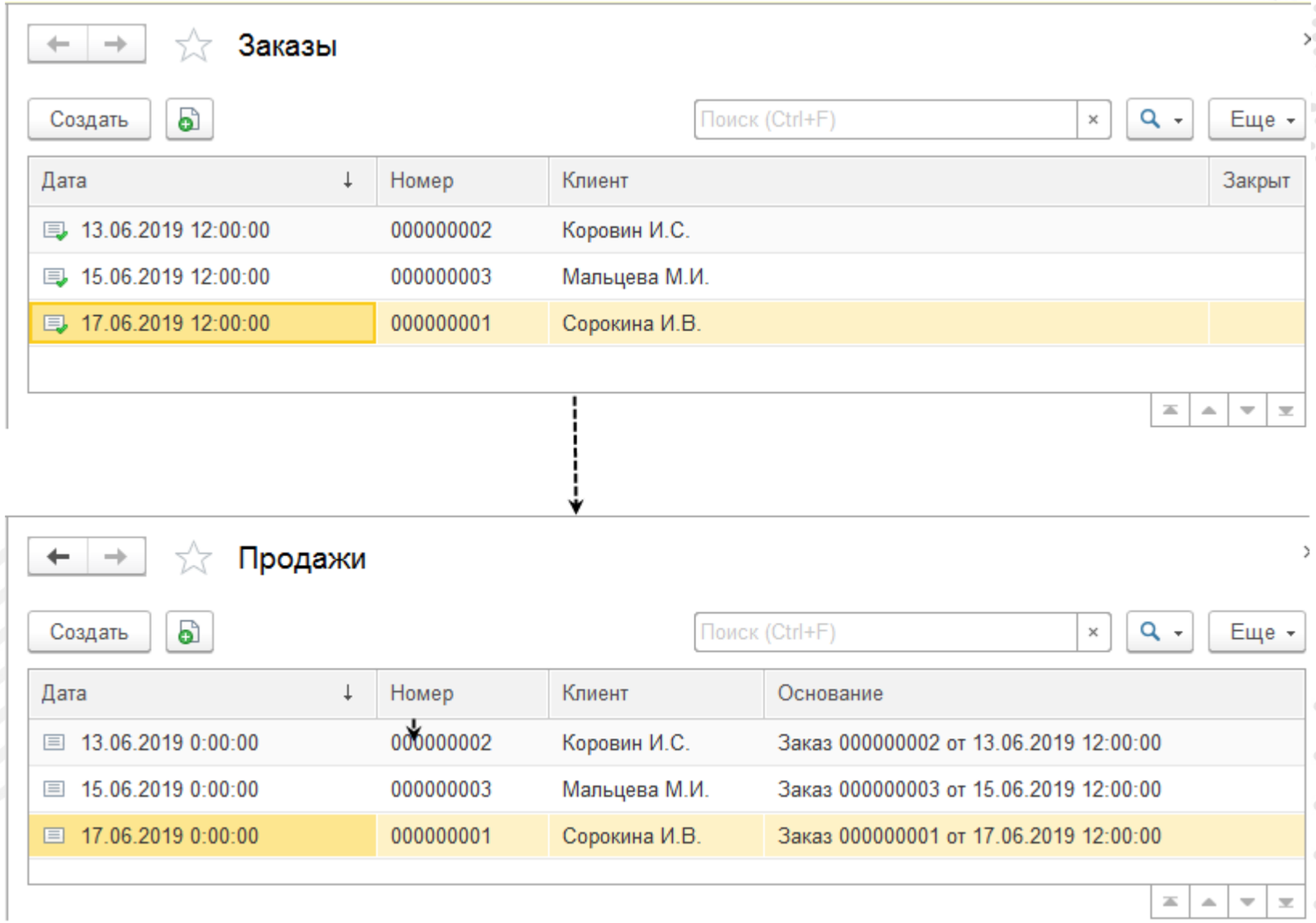
Пример реализации Web-сервиса

На основе статической ссылки на Web-сервис создается объект WSПрокси, с помощью которого выполняется операция Web-сервиса ПолучитьНезакрытыеЗаказы().

В результате список незакрытых заказов возвращается Web-сервисом в виде списка XDTO в переменную СписокЗаказов.

Далее в цикле обхода этого списка (СписокЗаказов.Список) на основании каждого заказа создается расходная накладная, и ее реквизиты: Дата, Клиент, Основание, а также табличная часть Товары заполняются данными заказа.

Пример реализации Web-сервиса



Пример реализации Web-сервиса

Таким образом, в прикладном решении потребителя Web-сервиса появятся расходные накладные, соответствующие заказам товаров, которые существуют у поставщика этого Web-сервиса.

